



Software Factory en la Era de la IA

DevOps Infra

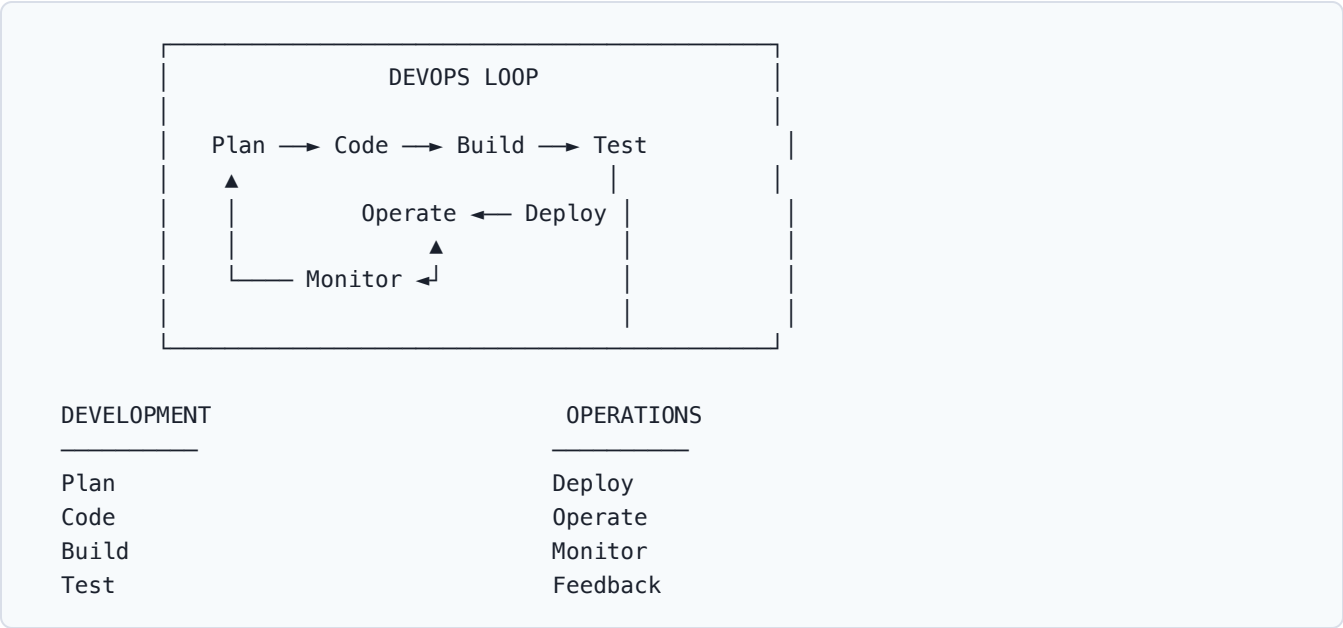
Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Devops Ciclo

El Ciclo de DevOps

DevOps es un ciclo continuo de retroalimentación que integra desarrollo y operaciones en un flujo sin fin. Este artículo profundiza en el **infinity loop**, el framework **CALMS** y las prácticas que lo sustentan.

El Infinity Loop

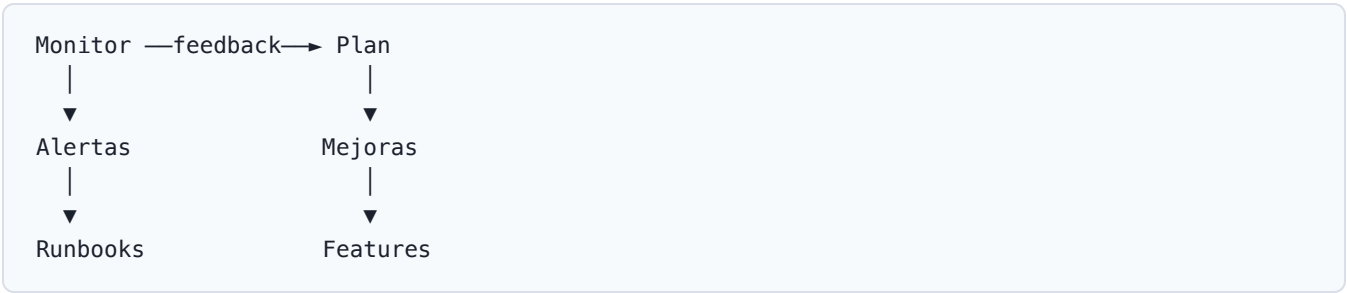


Las 8 fases del ciclo

Fase	Descripción	Herramientas típicas
Plan	Planificación del trabajo	Jira, Linear, GitHub Issues
Code	Escritura y revisión de código	Git, GitHub, GitLab
Build	Compilación y empaquetado	Maven, npm, Gradle
Test	Pruebas automatizadas	JUnit, Selenium, Cypress
Release	Gestión de versiones	Semantic Release, Versioning
Deploy	Despliegue a producción	Kubernetes, ArgoCD, Terraform
Operate	Ejecución en producción	Docker, cloud providers
Monitor	Observabilidad y alertas	Prometheus, Grafana, Datadog

Feedback loops

El ciclo no es lineal — cada fase retroalimenta a las anteriores:



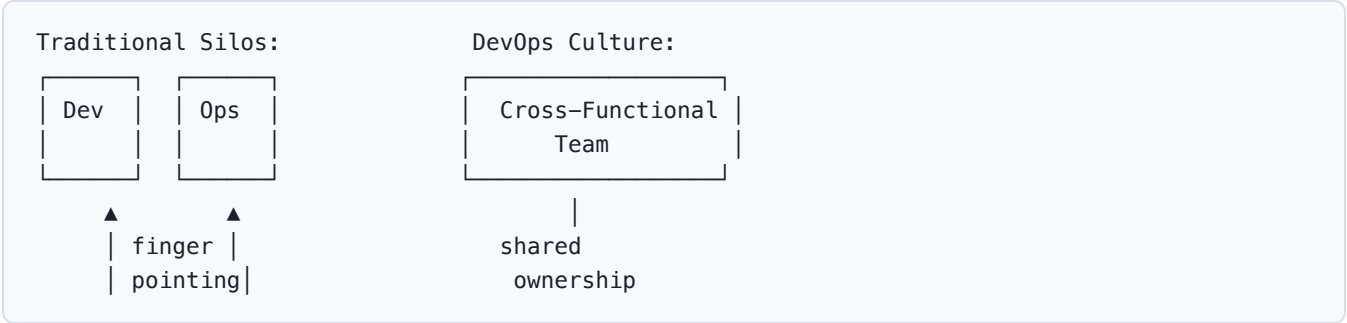
Tipos de feedback:

- **Fast feedback:** build y test en minutos (CI)
- **Medium feedback:** deploy a staging, QA automático
- **Slow feedback:** métricas de producción, NPS, usage analytics

CALMS: El framework de DevOps

C — Culture (Cultura)

La cultura DevOps se basa en la colaboración, la confianza y la responsabilidad compartida.



Prácticas culturales:

- **Blameless post-mortems:** analizar sistemas, no personas
- **Shared on-call:** developers participan en rotación
- **ChatOps:** incidentes y operaciones en canales compartidos
- **Game days:** simulacros de fallos controlados

A — Automation (Automatización)

Área	Qué automatizar	Impacto
Build	Compilación, linting, empaquetado	Consistencia, velocidad
Test	Unit, integration, e2e, security	Calidad, confianza

Área	Qué automatizar	Impacto
Deploy	Blue-green, canary, rolling updates	Reducción de riesgo
Infra	Provisionamiento, configuración	Reproducibilidad
Monitor	Alertas, dashboards, incidentes	Visibilidad

Niveles de automatización:

1. **Nivel 0:** Manual — builds y deploys manuales
2. **Nivel 1:** Scripts — bash scripts repetitivos
3. **Nivel 2:** CI básico — build automático en commit
4. **Nivel 3:** CD avanzado — deploy automático con quality gates
5. **Nivel 4:** Auto-remediation — corrección automática de fallos

L — Lean (Pensamiento Lean)

Derivado de Lean Software Development:

- **Eliminar desperdicio:** código muerto, contexto innecesario
- **Amplificar aprendizaje:** experimentos rápidos
- **Decidir lo más tarde posible:** opciones flexibles
- **Entregar lo más rápido posible:** batch pequeño
- **Empoderar al equipo:** autonomous teams
- **Optimizar el todo:** no局部 optimizar

Kanban en DevOps:

To Do	In Progress	Code Review	Testing	Done	Deployed
	WIP=3	WIP=2	WIP=3		

M — Measurement (Medición)

Las DORA Metrics son el estándar:

Métrica	Elite	High	Medium	Low
Deployment Frequency	On-demand	Weekly-monthly	Monthly-halfly	Halfly-yearly
Lead Time	< 1 day	1 day-1 week	1 week-1 month	> 6 months
Change Failure Rate	0-15%	16-30%	16-30%	> 6 months
MTTR	< 1 hour	< 1 day	1 week-1 month	> 6 months

Métricas complementarias:

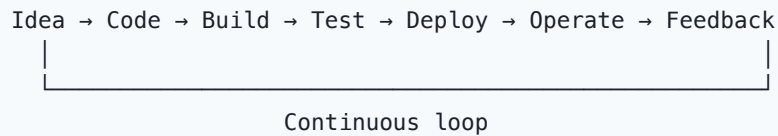
- **Cycle Time:** tiempo de desarrollo por feature
- **Code Coverage:** cobertura de tests

- **Defect Escape Rate:** bugs que llegan a producción
- **Toil Ratio:** % de trabajo manual vs automático

S — Sharing (Conocimiento Compartido)

Práctica	Descripción
Documentation as Code	Docs en repo, versionados con el código
Internal Tech Talks	Sesiones de conocimiento entre equipos
Pair/Mob Programming	Programación colaborativa
Runbooks compartidos	Guías de operación estandarizadas
ChatOps	Incidentes y decisiones en canales abiertos
Blameless culture	Aprendizaje sin culpables

DevOps y la cadena de valor



Relación con Value Stream Mapping:

- **Lead Time Total:** idea hasta valor en producción
- **Process Time:** tiempo de transformación real
- **% C&A:** % de actividad sin rework

Anti-patrones en DevOps

Anti-patrón	Señales	Solución
DevOps Team	Equipo dedicado a DevOps	Embed en equipos productivos
Tool obsession	Compran herramientas sin cultura	Cultura primero, herramientas después
Silo automation	CI/CD automatizado pero no colaboración	Romper silos con prácticas compartidas
No measurement	"Hacemos DevOps" sin métricas	DORA metrics, dashboards
Blame culture	Post-mortems que buscan culpables	Blameless post-mortems

DevOps en la Software Factory

La Software Factory implementa DevOps como:

1. **Pipeline estándar:** todos los proyectos usan el mismo flujo

2. **Herramientas compartidas**: centro de plataforma
3. **Métricas unificadas**: dashboards consolidados
4. **Gobernanza automatizada**: quality gates
5. **Onboarding acelerado**: playbooks de DevOps

Relación con otros conceptos

- DevOps como Filosofía — Fundamentos teóricos
- Manifiesto Ágil — Raíces del mindset
- Lean Software — Eliminar desperdicio
- CI/CD Pipeline — Implementación práctica
- DORA Metrics — Cómo medir DevOps
- Plataformas de Ingeniería — Internal Developer Platforms
- GitOps — Flujo declarativo basado en Git

Continuous Delivery

Continuous Delivery

Continuous Delivery (CD) es la capacidad de entregar cambios de software de forma frecuente, segura y sostenible a producción. Basado en los principios de Jez Humble y David Farley.

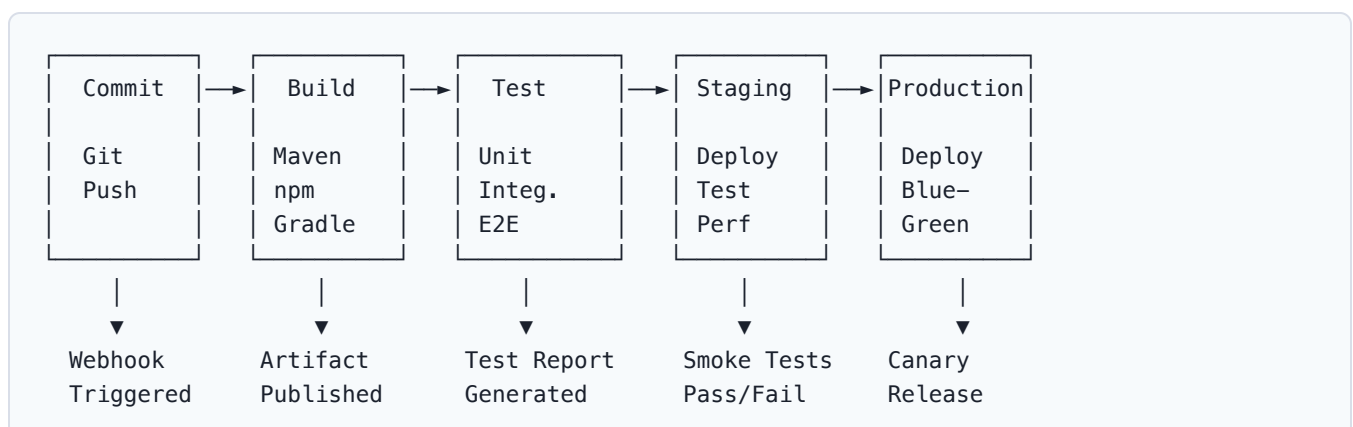
Principios fundamentales

"Continuous Delivery is the ability to get changes of all types — including new features, configuration changes, bug fixes and experiments — into production or into the hands of users safely and quickly in a sustainable way."
— Jez Humble

Los 10 principios de CD

1. **Repeatable and reliable process**: el proceso de deploy debe ser automatizado y repetible
2. **Automatizar casi todo**: build, test, deploy, infrastructure
3. **Keep everything in version control**: código, configs, scripts, infrastructure
4. **Done means deployed**: done implica en producción
5. **Test, test, test**: test pyramid, contract testing
6. **Deploy often**: frecuencia reduce riesgo
7. **Build quality in**: quality gates, no gates manuales
8. **Everyone can see**: transparencia en el pipeline
9. **Improve regularly**: retrospectives, métricas

Deployment Pipeline



Fases del pipeline

Fase	Qué ejecuta	Criterio de calidad
Commit	Build, unit tests, lint	0 errores, tests pasan
Artifact	Package, docker build	Artifact publicado
Integration	API tests, contract tests	Contratos válidos
E2E	UI tests, smoke tests	Flujos críticos OK
Staging	Deploy a environment igual a prod	Deploy exitoso
Performance	Load tests, stress tests	SLAs de performance
Production	Blue-green o canary deploy	Health checks OK

Quality Gates

```
# Ejemplo de quality gates en pipeline
stages:
  - build:
    gate:
      min_coverage: 80
      max_vulnerabilities: 0
      lint_errors: 0

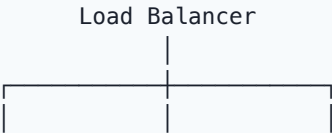
  - integration:
    gate:
      all_tests_pass: true
      contract_compatibility: true

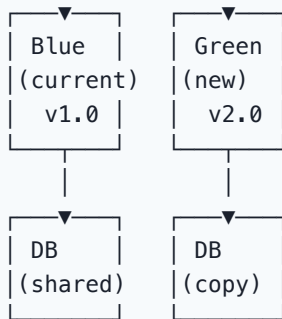
  - staging:
    gate:
      smoke_tests: pass
      performance_regression: "< 10%"

  - production:
    gate:
      canary_error_rate: "< 1%"
      canary_latency_p99: "< 500ms"
```

Estrategias de deployment

Blue-Green Deployment





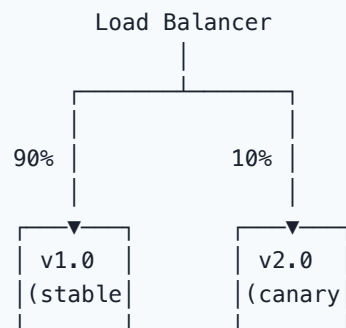
Steps:

1. Deploy v2.0 to Green
2. Run tests on Green
3. Switch LB to Green
4. Keep Blue as rollback

Ventajas: rollback instantáneo, zero downtime

Desventajas: duplica infraestructura, migration de DB compleja

Canary Release



Steps:

1. Deploy v2.0 a 10% del tráfico
2. Monitor error rate, latency
3. Si OK, incrementar gradualmente
4. Si error, rollback automático
5. 100% → v2.0 es el nuevo stable

Ventajas: riesgo reducido, feedback real

Desventajas: complejo de implementar, necesita feature flags

Rolling Update (Kubernetes)

```

apiVersion: apps/v1
kind: Deployment
spec:

```

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1    # Máximo 1 pod no disponible
    maxSurge: 1          # Máximo 1 pod extra
template:
  spec:
    containers:
      - name: app
        image: myapp:v2.0
        readinessProbe:
          httpGet:
            path: /health
            port: 8080
          initialDelaySeconds: 5
          periodSeconds: 5
```

Continuous Delivery vs Continuous Deployment

Aspecto	Continuous Delivery	Continuous Deployment
Deploy a prod	Automatizado pero manual trigger	Automático en cada commit
Aprobación	Manual gate antes de prod	Automática con quality gates
Frecuencia	Cuando se decida	En cada cambio aprobado
Riesgo	Bajo (tested)	Muy bajo (small batches)
Requisito	Pipeline completo	Pipeline + testing completo

```
graph LR
    CI[Continuous Integration] --> CD[Continuous Delivery]
    CD --> CP[Continuous Deployment]
    CI -- Build + Test --> CI
    CD -- "+ Deploy to Staging (manual)" --> CD
    CP -- "+ Deploy to Prod (automatic)" --> CP
```

Feature Flags

```
// Feature flag para deployment seguro
const config = {
  newCheckout: {
    enabled: true,
    percentage: 25,
    whitelist: ['beta-users@company.com']
  }
};

function checkout(user) {
  if (config.newCheckout.enabled) {
```

```
    if (isInRolloutGroup(user, config.newCheckout)) {  
        return newCheckoutFlow(user);  
    }  
}  
return legacyCheckout(user);  
}
```

Tipo	Uso
Release	Rollout gradual de features
Experiment	A/B testing
Ops	Kill switches, circuit breakers
Permission	Feature gating por plan

Métricas de CD

Relacionado con DORA Metrics:

Métrica	Objetivo CD
Lead Time for Changes	< 1 hora
Deployment Frequency	Múltiples veces al día
Change Failure Rate	< 5%
MTTR	< 1 hora
Pipeline Duration	< 30 minutos
Time to Restore	< 15 minutos

Herramientas de CD

Categoría	Herramientas	
CI/CD Platform	Jenkins, GitHub Actions, GitLab CI, CircleCI	
GitOps	ArgoCD, Flux (ver [../09-DevOps-Infra/07-gitops	GitOps]])
Deploy	Spinnaker, Harness, Octopus Deploy	
Feature Flags	LaunchDarkly, Unleash, Flagsmith	
Artifact	Nexus, Artifactory, GitHub Packages	

Anti-patrones

Anti-patrón	Problema	Solución
Deploy by hand	Deploy manual cada vez	Automatizar pipeline
Long-lived branches	Branches que viven semanas	Trunk-based development
Flaky tests	Tests que fallan intermitentemente	Fix o quarantinar
Big-bang release	Months of work, one release	Small, frequent releases
No rollback plan	Deploy sin plan de reversión	Blue-green, canary

Relación con otros conceptos

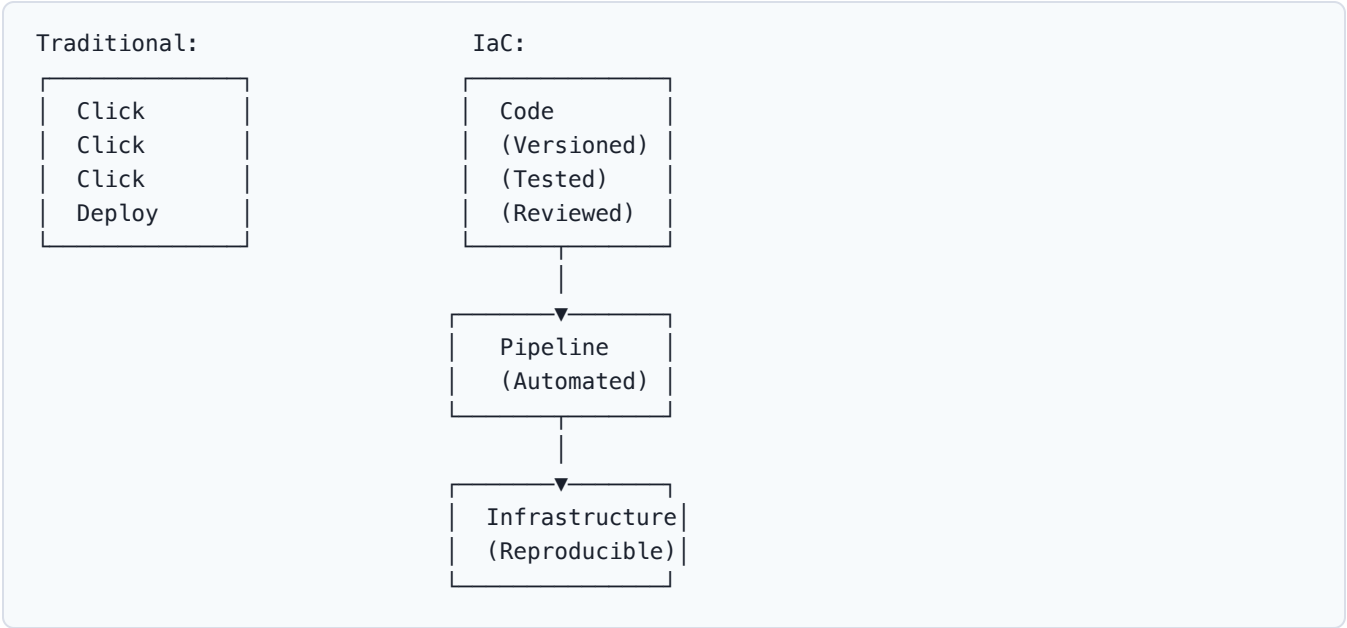
- El Ciclo de DevOps — Framework CALMS
- CI/CD Pipeline — Implementación técnica
- Trunk-Based Development — Branching strategy para CD
- GitOps — CD declarativo basado en Git
- Quality Gates — Criterios de calidad
- SRE — Deployment con error budgets
- Deployment Frequency — Métrica clave

Infrastructure As Code

Infrastructure as Code

Infrastructure as Code (IaC) es la práctica de gestionar y provisionar infraestructura de computación a través de archivos de configuración en lugar de procesos manuales o scripts ad-hoc.

¿Qué es IaC?



Beneficios de IaC

Beneficio	Descripción
Reproducibilidad	Mismo resultado cada vez
Version control	Historial de cambios en infra
Colaboración	PRs para cambios de infra
Testing	Validar infra como código
Self-documenting	El código documenta la infra
Speed	Provisionar en minutos, no días

Modelos de IaC

Declarativo vs Imperativo

Declarativo (qué quieres):

```
# Terraform - Declarativo
resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbfafa1f0"
  instance_type = "t3.micro"

  tags = {
    Name = "web-server"
  }
}
```

Imperativo (cómo hacerlo):

```
# Pulumi - Imperativo (también soporta declarativo)
import pulumi
import pulumi_aws as aws

web = aws.ec2.Instance("web",
  ami="ami-0c55b159cbfafa1f0",
  instance_type="t3.micro",
  tags={"Name": "web-server"})
```

Approaches de IaC

IaC Approaches		
Push-based	Pull-based	Agent-based
Terraform CloudFormation Pulumi	Ansible SaltStack	Puppet Chef
"Push config to servers"	"Pull config from master"	"Agent maintains desired state"

Terraform

Conceptos básicos

```
# Provider configuration
provider "aws" {
  region = "us-east-1"
}
```

```

# Resource definition
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "main-vpc"
  }
}

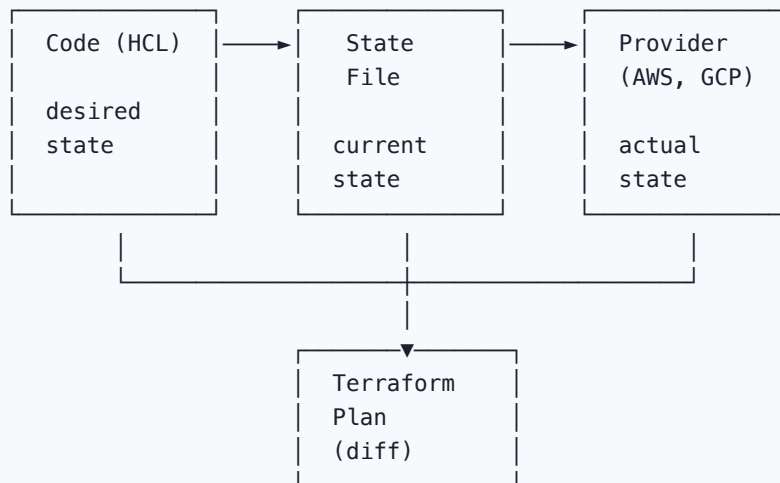
# Data source
data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

# Output
output "vpc_id" {
  value = aws_vpc.main.id
}

```

State Management



Remote State (recomendado):

```

terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}

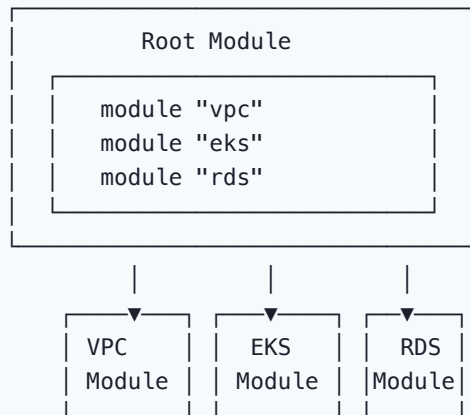
```

```

    dynamodb_table = "terraform-locks"
    encrypt          = true
  }
}

```

Módulos



```

module "vpc" {
  source = "../modules/vpc"

  cidr_block = "10.0.0.0/16"
  azs        = ["us-east-1a", "us-east-1b"]

  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]
}

module "eks" {
  source = "../modules/eks"

  vpc_id      = module.vpc.vpc_id
  subnet_ids  = module.vpc.private_subnets
}

```

Ansible

Configuration Management

```

# playbook.yml
---
- hosts: webservers
  become: yes
  vars:

```

```

http_port: 80
app_version: "2.1.0"

tasks:
  - name: Install dependencies
    apt:
      name:
        - nginx
        - python3
      state: present
      update_cache: yes

  - name: Deploy application
    copy:
      src: "dist/app-{{ app_version }}.tar.gz"
      dest: /opt/app/
    notify: Restart app

  - name: Configure nginx
    template:
      src: nginx.conf.j2
      dest: /etc/nginx/nginx.conf
    notify: Reload nginx

handlers:
  - name: Restart app
    service:
      name: myapp
      state: restarted

  - name: Reload nginx
    service:
      name: nginx
      state: reloaded

```

Terraform vs Ansible

Aspecto	Terraform	Ansible
Type	Provisioning	Configuration Management
State	Manages state	Agentless, stateless
Language	HCL (declarative)	YAML (procedural)
Use case	Create infrastructure	Configure infrastructure
Cloud	Multi-cloud	Multi-platform
Idempotency	Yes	Yes

Pulumi

```
// Pulumi - TypeScript
import * as pulumi from "@pulumi/pulumi";
import * as aws from "@pulumi/aws";

const bucket = new aws.s3.Bucket("my-bucket", {
    tags: {
        Environment: "dev",
    },
});

const role = new aws.iam.Role("role", {
    assumeRolePolicy: JSON.stringify({
        Version: "2012-10-17",
        Statement: [{
            Action: "sts:AssumeRole",
            Effect: "Allow",
            Principal: {
                Service: "lambda.amazonaws.com",
            },
        }],
    }),
});

exports.bucketName = bucket.id;
exports.roleArn = role.arn;
```

Testing Infrastructure

```
# Terraform validate y plan
terraform validate
terraform plan -out=tfplan

# Terratest (Go)
func TestVpcModule(t *testing.T) {
    terraformOptions := terraform.WithDefaultRetryableErrors(t, &terraform.Options{
        TerraformDir: "../modules/vpc",
        Vars: map[string]interface{}{
            "cidr_block": "10.0.0.0/16",
        },
    })
    defer terraform.Destroy(t, terraformOptions)
    terraform.InitAndApply(t, terraformOptions)

    vpcId := terraform.Output(t, terraformOptions, "vpc_id")
    assert.Regexp(t, `^vpc-`, vpcId)
}
```

Mejores prácticas

Práctica	Descripción
Remote state	Nunca commit local state
State locking	DynamoDB para Terraform
Modules	Reutilizar patrones comunes
Environments	Separate state per env
Drift detection	Detectar cambios manuales
Plan before apply	Siempre revisar diff
Secrets management	Vault, SSM, never in code
Code review	PRs para cambios de infra

IaC y GitOps



Ver GitOps para el flujo declarativo completo.

Relación con otros conceptos

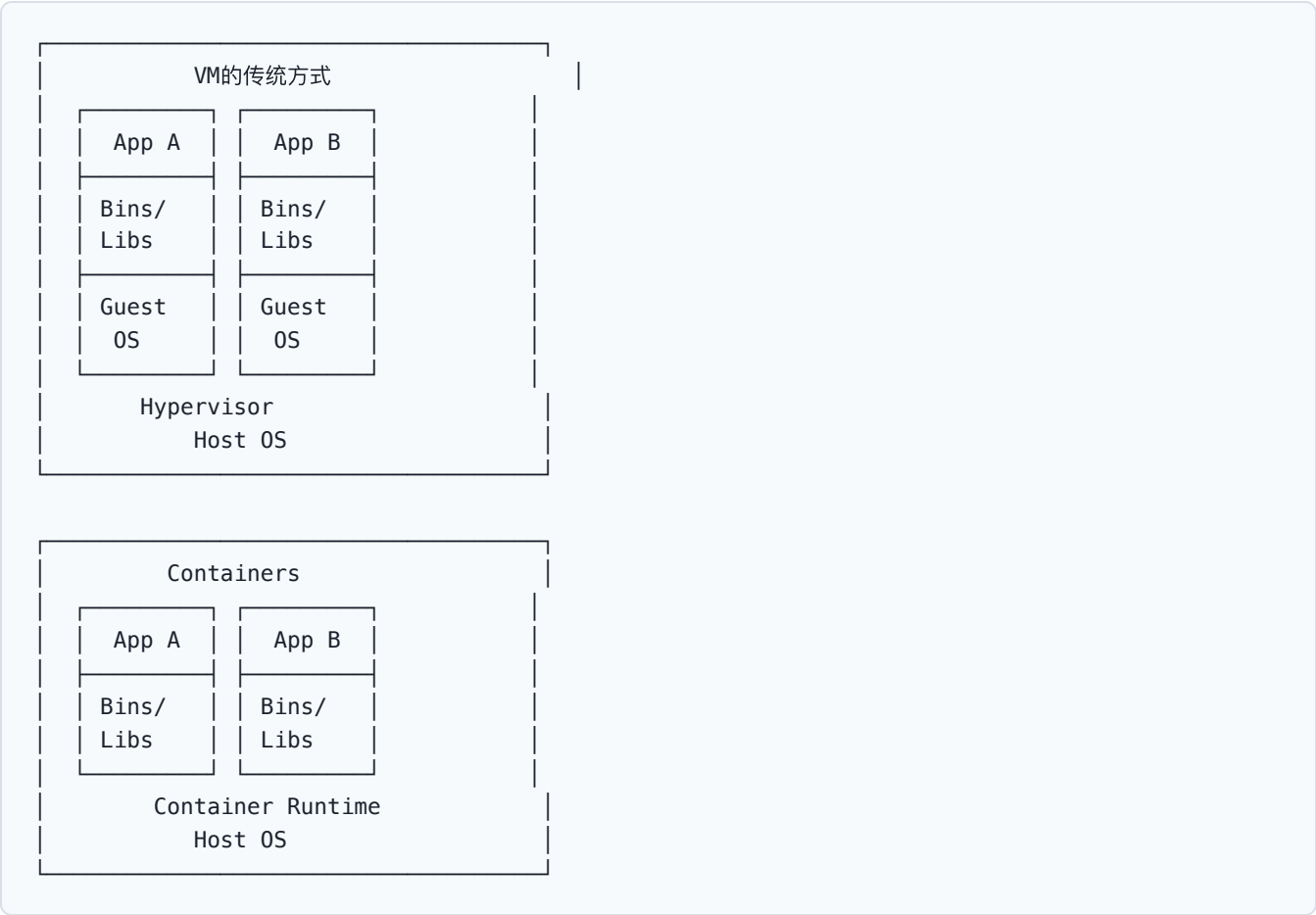
- Containers y Docker — Infraestructura para containers
- Kubernetes — Orquestación declarativa
- GitOps — Flujo GitOps para infra
- Terraform en Herramientas — Evaluación de herramientas
- Supply Chain Security — Seguridad en dependencias
- Optimización de Costos — Cost awareness en IaC

Containers Docker

Containers y Docker

Los containers empaquetan una aplicación con todas sus dependencias en una unidad estandarizada que se ejecuta de forma consistente en cualquier entorno.

¿Qué es un Container?



Característica	VM	Container
Isolation	Full OS	Process-level
Size	GBs	MBs
Boot time	Minutes	Seconds
Overhead	High	Low
Density	Few per host	Many per host

Característica	VM	Container
Portability	Limited	High

Dockerfile

Estructura básica

```
# Stage 1: Build
FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

COPY . .
RUN npm run build

# Stage 2: Production
FROM node:18-alpine AS production

# Security: non-root user
RUN addgroup -g 1001 -S appgroup && \
    adduser -S appuser -u 1001 -G appgroup

WORKDIR /app

# Copy only production artifacts
COPY --from=builder --chown=appuser:appgroup /app/dist ./dist
COPY --from=builder --chown=appuser:appgroup /app/node_modules ./node_modules

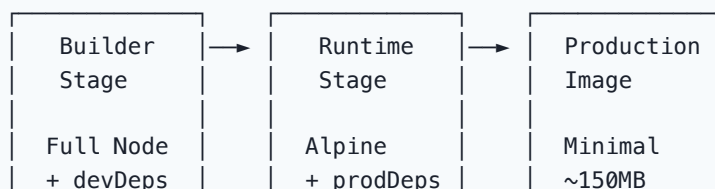
USER appuser

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

CMD ["node", "dist/main.js"]
```

Multi-stage builds



+ build
~1.2GB

+ dist
~200MB

Best practices

Práctica	Ejemplo
Multi-stage	Reducir tamaño de imagen
Alpine base	FROM node:18-alpine
Non-root user	USER appuser
COPY over ADD	COPY para archivos simples
Order layers	Dependencias primero, código después
.dockerignore	Excluir node_modules, .git
HEALTHCHECK	Verificar salud del contenedor
Pin versions	node:18.17.1-alpine no latest

.dockerignore

```
node_modules
npm-debug.log
.git
.env
.env.local
dist
coverage
.github
*.md
```

Docker Compose

```
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
```

```
- DATABASE_URL=postgresql://postgres:password@db:5432/myapp
- REDIS_URL=redis://cache:6379
depends_on:
  db:
    condition: service_healthy
  cache:
    condition: service_healthy
volumes:
  - ./uploads:/app/uploads
networks:
  - app-network
restart: unless-stopped

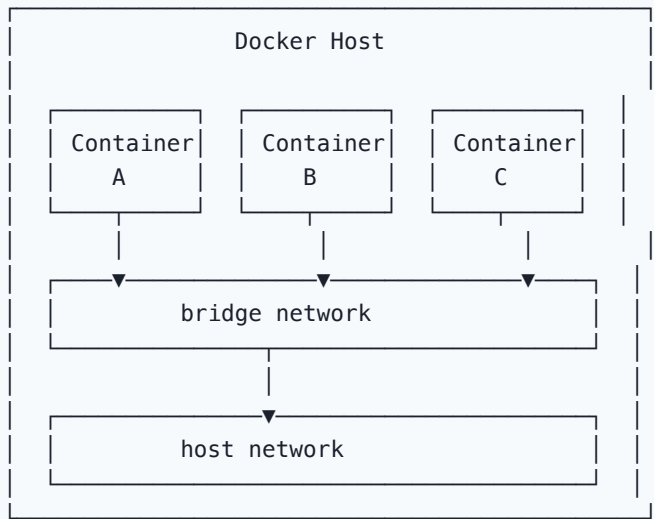
db:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: myapp
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
  volumes:
    - postgres_data:/var/lib/postgresql/data
  ports:
    - "5432:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - app-network

cache:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
  networks:
    - app-network

volumes:
  postgres_data:

networks:
  app-network:
    driver: bridge
```

Docker Networking



Network	Descripción
bridge	Default. Containers se comunican por IP
host	Container usa red del host
overlay	Multi-host networking (Swarm)
none	Sin red
macvlan	Asigna MAC address al container

Docker Volumes

```
# Named volume
docker volume create mydata
docker run -v mydata:/app/data myimage

# Bind mount
docker run -v /host/path:/container/path myimage

# TMPFS (memory only)
docker run --tmpfs /app/temp myimage
```

Tipo	Persistencia	Uso
Named volume	Sí	Databases, data persistente
Bind mount	Sí	Desarrollo, config files

Tipo	Persistencia	Uso
tmpfs	No	Secrets, temp files

Comandos esenciales

```
# Build
docker build -t myapp:1.0 .
docker build --no-cache -t myapp:1.0 .

# Run
docker run -d -p 3000:3000 --name myapp myapp:1.0
docker run -it --rm alpine sh

# Manage
docker ps -a
docker logs -f myapp
docker exec -it myapp sh
docker stats

# Cleanup
docker system prune -a
docker volume prune
docker image prune -a
```

Seguridad en containers

Práctica	Descripción
Non-root user	USER appuser en Dockerfile
Read-only FS	--read-only flag
No new privileges	--security-opt=no-new-privileges
Scan images	Trivy, Snyk, Clair
Minimal base	Alpine, distroless, scratch
Pin versions	No usar latest
Secrets	Docker secrets, not ENV

```
# Scan image for vulnerabilities
trivy image myapp:1.0

# Run with security options
docker run --read-only \
  --security-opt=no-new-privileges \
  --cap-drop=ALL \
```

```
--cap-add=NET_BIND_SERVICE \  
myapp:1.0
```

Containers y Cloud Native

Relacionado con Cloud Native Architecture:

- Containers son el building block de cloud native
- Kubernetes orquesta containers a escala
- Kubernetes manage containers en clusters

Relación con otros conceptos

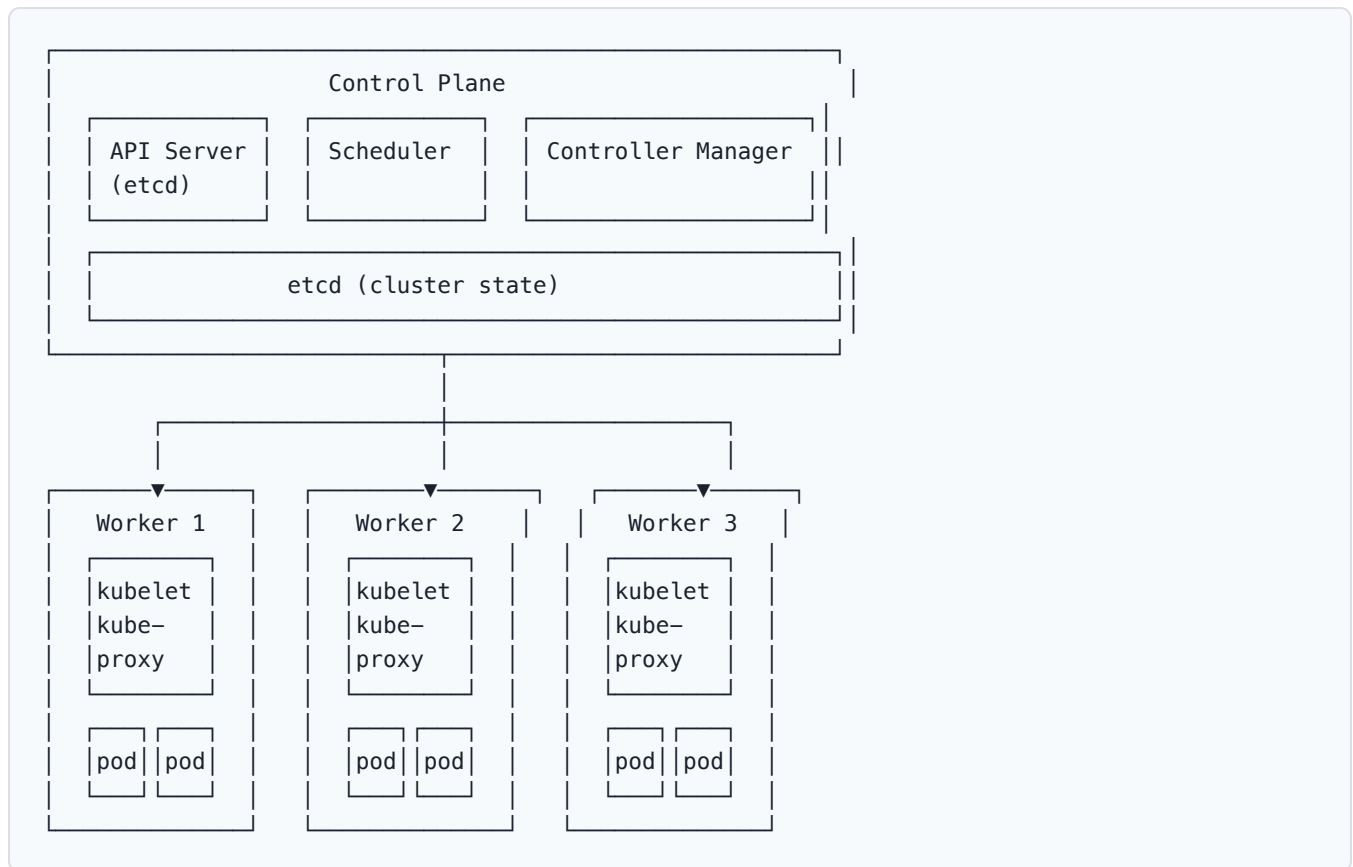
- Kubernetes — Orquestación de containers
- IaC — Infraestructura para containers
- Containers en Herramientas — Evaluación de herramientas
- Microservicios — Containers habilitan microservicios
- Cloud Native — Arquitectura cloud native
- DevSecOps — Seguridad en containers

Kubernetes Orchestration

Kubernetes y Orquestación

Kubernetes (K8s) es la plataforma de orquestación de containers de facto para desplegar, escalar y gestionar aplicaciones containerizadas en clusters distribuidos.

Arquitectura de Kubernetes



Core Resources

Pods

La unidad más pequeña desplegable en K8s:

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
```

```
    tier: backend
spec:
  containers:
  - name: myapp
    image: myapp:1.0
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "128Mi"
        cpu: "250m"
      limits:
        memory: "256Mi"
        cpu: "500m"
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 10
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
```

Deployments

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
```

```
image: myapp:2.0
ports:
  - containerPort: 8080
env:
  - name: DATABASE_URL
    valueFrom:
      secretKeyRef:
        name: db-secret
        key: url
```

Services

```
# ClusterIP (interno)
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: ClusterIP

---
# NodePort (exposición externa simple)
apiVersion: v1
kind: Service
metadata:
  name: myapp-nodeport
spec:
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30080
  type: NodePort

---
# LoadBalancer (cloud)
apiVersion: v1
kind: Service
metadata:
  name: myapp-lb
spec:
  selector:
    app: myapp
```

```
ports:
  - port: 80
    targetPort: 8080
type: LoadBalancer
```

Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  tls:
    - hosts:
        - app.example.com
      secretName: app-tls
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: myapp-service
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 8080
```

ConfigMaps y Secrets

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  config.json: |
```

```

    {
      "feature_flags": {
        "new_ui": true
      }
    }
  }
}

---
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  url: cG9zdGdyZXNxbDovL3VzZXI6cGFzc0Bob3N00jU0MzIvZGI=
  password: cGFzc3dvcmQxMjM=

```

Horizontal Pod Autoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80

```

Helm

Conceptos

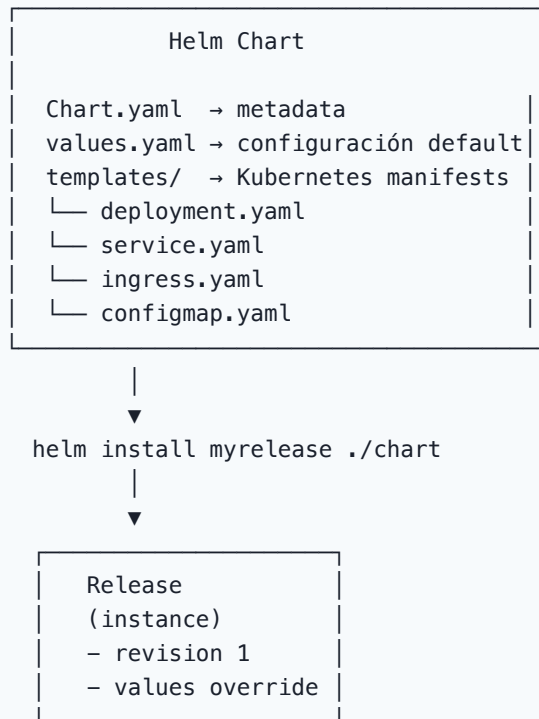


Chart example

```
# Chart.yaml
apiVersion: v2
name: myapp
description: My application Helm chart
version: 1.0.0
appVersion: "2.0.0"
dependencies:
  - name: postgresql
    version: "12.x.x"
    repository: "https://charts.bitnami.com/bitnami"
```

```
# values.yaml
replicaCount: 2

image:
  repository: myapp
  tag: "2.0.0"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80
```

```
ingress:
  enabled: true
  hosts:
    - host: app.example.com
      paths:
        - path: /
          pathType: Prefix

resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "200m"
```

Helm commands

```
# Install
helm install myrelease ./chart -f values-prod.yaml

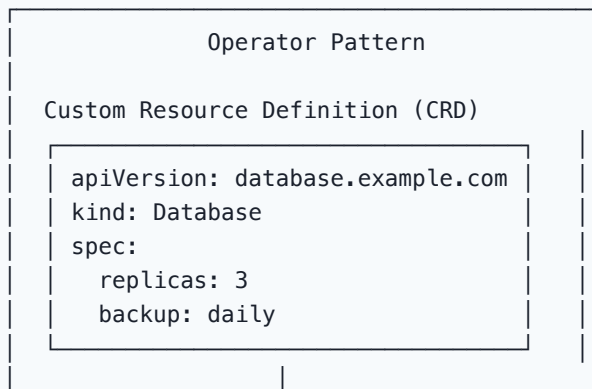
# Upgrade
helm upgrade myrelease ./chart --set image.tag=2.1.0

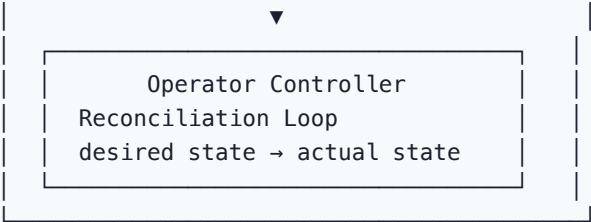
# Rollback
helm rollback myrelease 1

# List releases
helm list -A

# Debug
helm template myrelease ./chart
helm diff upgrade myrelease ./chart
```

Operators





▼

Operator Controller
Reconciliation Loop
desired state → actual state

The diagram shows a light blue rounded rectangle containing a dashed-line box. Inside the dashed box, the text 'Operator Controller' is at the top, followed by 'Reconciliation Loop' and 'desired state → actual state' on separate lines. A small downward-pointing triangle is positioned above the dashed box.

Operators populares:

- Prometheus Operator
- Cert-Manager
- ArgoCD Operator
- PostgreSQL Operator (Zalando, CrunchyData)

Namespace Strategy

```
cluster
├── kube-system
├── monitoring
│   ├── prometheus
│   └── grafana
├── production
│   ├── frontend
│   └── backend
├── staging
│   ├── frontend
│   └── backend
└── development
    ├── team-a
    └── team-b
```

Relación con otros conceptos

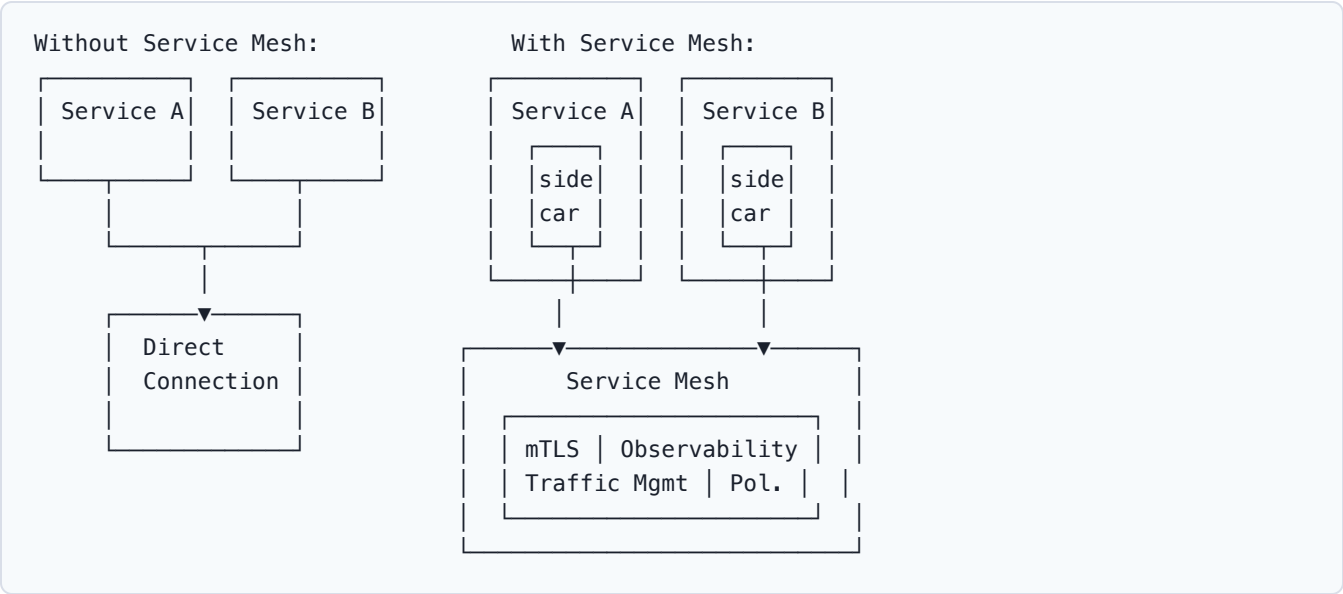
- Containers y Docker — Base de K8s
- Service Mesh — Red para microservicios en K8s
- IaC — Provisionar clusters
- GitOps — Deploy declarativo a K8s
- SRE — Operar K8s en producción
- K8s en Herramientas — Evaluación
- Cloud Native — K8s como plataforma cloud native

Service Mesh

Service Mesh

Service Mesh es una infraestructura de red dedicada para gestionar la comunicación entre microservicios, proporcionando seguridad, observabilidad y control de tráfico de forma transparente.

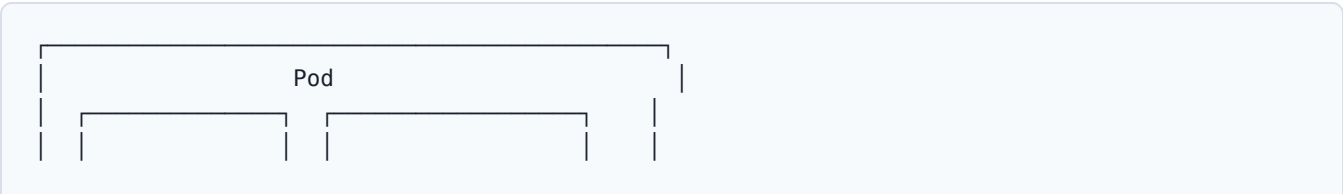
¿Qué es un Service Mesh?

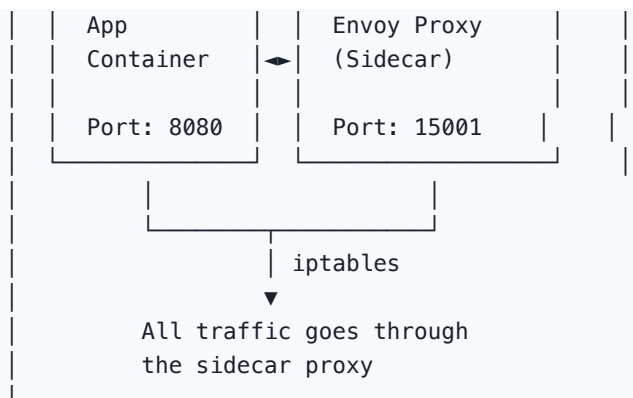


Capas del Service Mesh

Capa	Función	Ejemplo
Data Plane	Proxies que interceptan tráfico	Envoy, linkerd2-proxy
Control Plane	Configura los proxies	Istiod, Linkerd control plane
Observability	Métricas, tracing, logging	Jaeger, Prometheus, Kiali
Security	mTLS, authN/authZ	Cert-manager, OPA
Traffic Management	Routing, load balancing, retries	VirtualService, Gateway

Patrón Sidecar



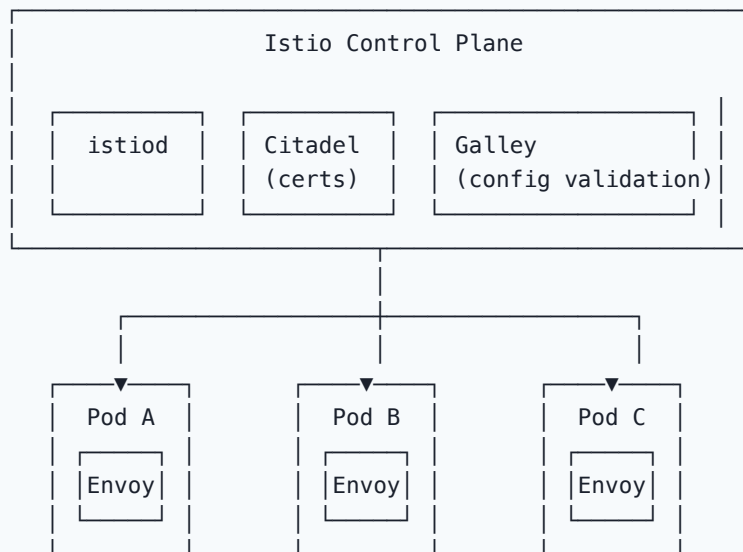


Ventajas del sidecar pattern:

- **Transparencia:** la app no necesita cambiar
- **Lenguaje agnostic:** funciona con cualquier stack
- **Separation of concerns:** networking separado de lógica de negocio
- **Upgrade independiente:** actualizar proxy sin redeploy app

Istio

Arquitectura



Traffic Management

```
# VirtualService - Routing rules
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
```

```

  name: myapp-vs
spec:
  hosts:
    - myapp
  http:
    - match:
        - headers:
            x-version:
              exact: "v2"
      route:
        - destination:
            host: myapp
            subset: v2
    - route:
        - destination:
            host: myapp
            subset: v1
            weight: 90
        - destination:
            host: myapp
            subset: v2
            weight: 10

```

```

---
# DestinationRule - Load balancing, circuit breaking
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: myapp-dr
spec:
  host: myapp
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 100
    http:
      h2UpgradePolicy: DEFAULT
      http1MaxPendingRequests: 100
      http2MaxRequests: 1000
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 30s
      baseEjectionTime: 3m
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2

```

Security (mTLS)

```
# PeerAuthentication – Enforce mTLS
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production
spec:
  mtls:
    mode: STRICT

---
# AuthorizationPolicy – Access control
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: myapp-policy
  namespace: production
spec:
  selector:
    matchLabels:
      app: myapp
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/production/sa/frontend"]
      to:
        - operation:
            methods: ["GET", "POST"]
            paths: ["/api/*"]
```

Linkerd

Comparación con Istio

Aspecto	Istio	Linkerd
Proxy	Envoy (C++)	linkerd2-proxy (Rust)
Complexity	Alta	Baja
Performance	Overhead moderado	Overhead mínimo
Features	Full-featured	Core features
Learning curve	Empinada	Suave
Resource usage	~100MB por proxy	~10MB por proxy

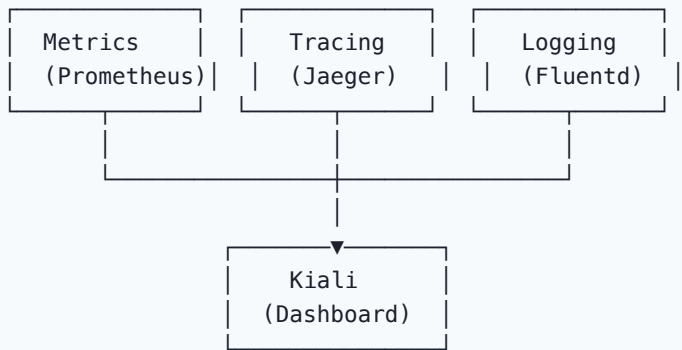
Linkerd installation

```
# Install Linkerd
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -

# Inject sidecar
kubectl get deployment myapp -o yaml | linkerd inject - | kubectl apply -f -

# Check health
linkerd check
linkerd viz install | kubectl apply -f -
linkerd viz dashboard
```

Observability con Service Mesh



Métricas que provee:

- **Request rate:** requests per second
- **Error rate:** 5xx errors percentage
- **Latency:** p50, p90, p99
- **Traffic:** bytes in/out
- **TCP connections:** active connections

Service Mesh: Cuándo usarlo

Señal	Acción
>10 microservicios	Considerar service mesh
mTLS requerido	Service mesh lo resuelve
Multi-language	Sidecar pattern es ideal
Observability limitada	Mesh provee métricas gratis

Señal	Acción
<5 servicios	Probablemente innecesario
Monolito	No usar

Relación con otros conceptos

- Kubernetes — Plataforma donde corre el mesh
- Cloud Native — Service mesh es patrón cloud native
- Microservicios — Mesh habilita microservicios
- SRE — Observabilidad y resiliencia
- Zero Trust — mTLS y authorization policies
- Incident Response — Traffic management durante incidentes



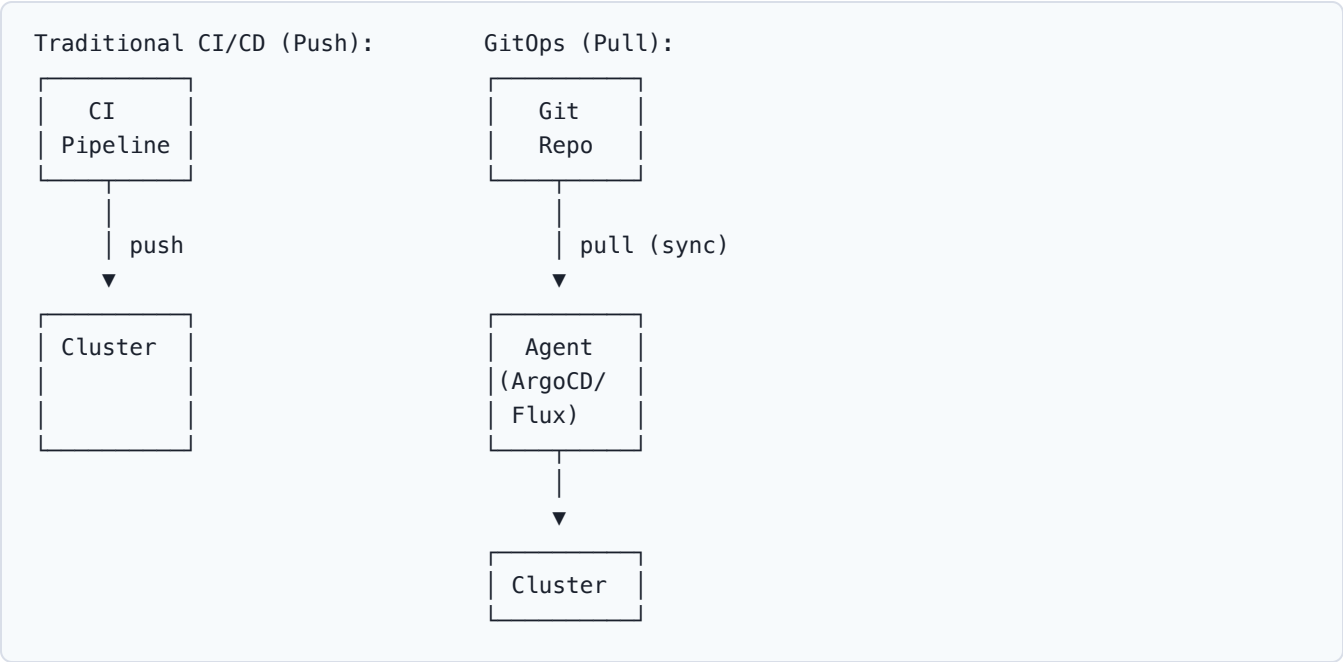
Gitops

GitOps como Práctica

GitOps es un modelo operativo donde Git es la fuente única de verdad para la infraestructura y las aplicaciones, declarando el estado deseado del sistema de forma versionada y auditada.

¿Qué es GitOps?

GitOps = IaC + Pull-based + Declarative + Versioned + Automated



Los 4 principios de GitOps

Principio	Descripción
Declarative	El estado deseado debe ser declarativo
Versioned	Todo está en Git, con historial completo
Automated	Los agents sincronizan automáticamente
Self-healing	El agent corrige drifts automáticamente

GitOps vs Traditional CI/CD

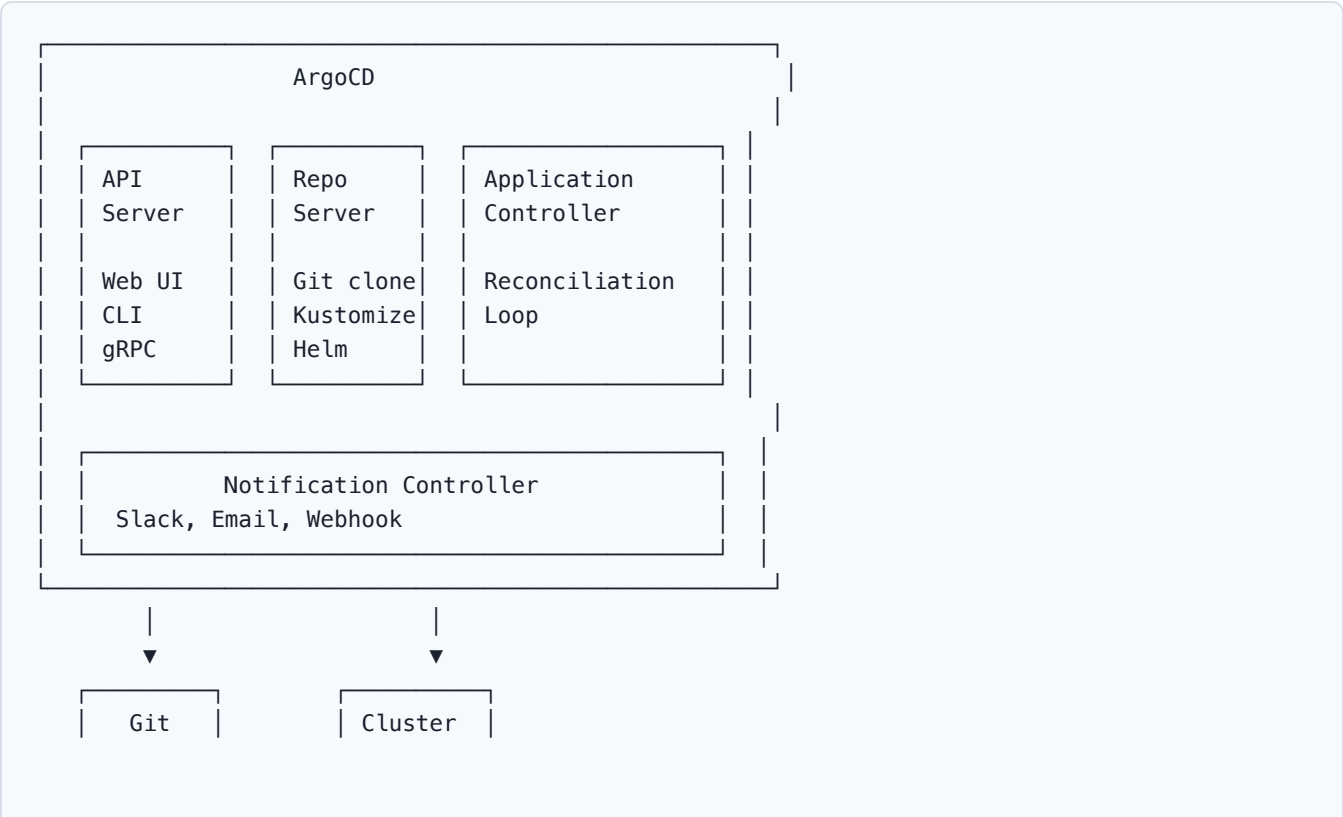
Aspecto	Traditional CI/CD	GitOps
Trigger	Push (pipeline ejecuta)	Pull (agent sincroniza)
Source of truth	Pipeline config	Git repository
Drift detection	Manual o no existe	Automático
Rollback	Re-run pipeline	git revert
Audit trail	Logs de pipeline	Git history
Security	Pipeline necesita acceso cluster	Agent tiene acceso limitado
Secrets	En pipeline variables	External secrets, sealed secrets

CI/CD Traditional:
Developer → Push → CI Pipeline → kubectl apply → Cluster

GitOps:
Developer → Push → Git Repo ← Sync ← ArgoCD → Cluster
(self-healing)

ArgoCD

Arquitectura



Repo

(target)

Application definition

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp-production
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/org/k8s-manifests.git
    targetRevision: main
    path: apps/myapp/overlays/production
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
      allowEmpty: false
    syncOptions:
      - CreateNamespace=true
      - PrunePropagationPolicy=foreground
  retry:
    limit: 5
    backoff:
      duration: 5s
      factor: 2
      maxDuration: 3m
```

Multi-environment promotion

Git Repo Structure:

```
|— apps/
|   |— myapp/
|   |   |— base/
|   |   |   |— deployment.yaml
|   |   |   |— service.yaml
|   |   |   |— kustomization.yaml
|   |   |— overlays/
|   |   |   |— dev/
|   |   |   |   |— kustomization.yaml
|   |   |   |   |— replicas-patch.yaml
|   |   |— staging/
```

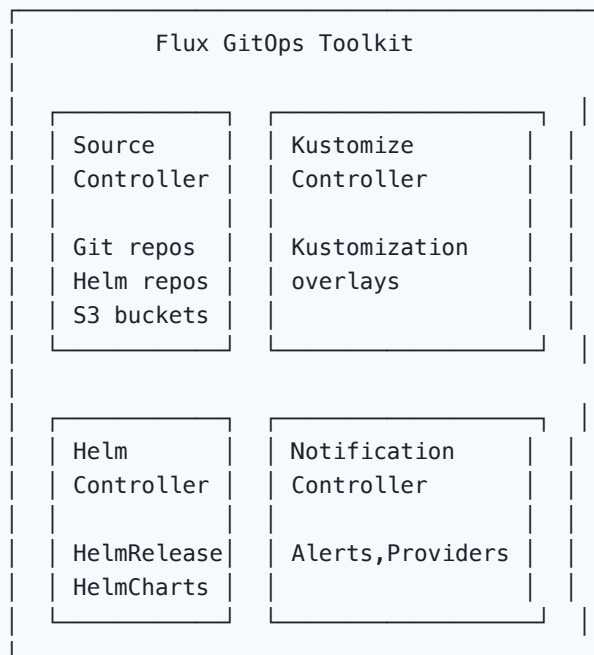


Promotion flow:

1. Developer push code
2. CI build & test
3. CI update image tag in overlays/dev
4. ArgoCD sync dev automatically
5. After QA approval → PR to staging
6. After staging validation → PR to production

Flux

Arquitectura Flux v2



Flux vs ArgoCD

Aspecto	Flux	ArgoCD
Architecture	Toolkit (múltiples controllers)	Monolithic
UI	Dashboard (weave-gitops)	Rich web UI

Aspecto	Flux	ArgoCD
Multi-tenancy	Namespace-based	Project-based
Helm support	Native HelmRelease	Via chart parameter
Image automation	Built-in	Via Image Updater
Learning curve	Moderate	Easier
CNCF Status	Graduated	Graduated

GitOps Maturity Model

Level 1: Git as backup

└─ kubectl apply + push to Git

Level 2: Pull-based deployment

└─ Agent syncs from Git (ArgoCD/Flux)

Level 3: Progressive delivery

└─ Canary, blue-green with GitOps

Level 4: Full GitOps

└─ Infrastructure + apps + secrets in Git

Level 5: Self-service platform

└─ Developers create apps via Git PRs

Secrets en GitOps

Enfoque	Descripción
Sealed Secrets	Encrypt secrets que solo el cluster puede decrypt
External Secrets	K8s secrets desde Vault, AWS SM, etc.
SOPS	Mozilla SOPS con age/KMS encryption
Vault	HashiCorp Vault con CSI driver

```
# External Secrets example
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-secret
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
```

```
kind: SecretStore
target:
  name: db-credentials
data:
  - secretKey: password
    remoteRef:
      key: prod/myapp/db
      property: password
```

Mejores prácticas

Práctica	Descripción
Repo structure	Base + overlays por ambiente
Branch strategy	Main branch, promotion via PRs
Automated sync	Auto-sync en dev/staging, manual en prod
Drift detection	Self-heal habilitado
Notifications	Alertas en sync failures
RBAC	Quién puede deployar a producción
Audit	Git history como audit trail

Relación con otros conceptos

- Continuous Delivery — CD push vs GitOps pull
- Kubernetes — Target platform
- IaC — Infra declarativa en Git
- Git Flow — Estrategia de branching
- Supply Chain Security — Seguridad en supply chain
- Optimización de Costos — Visibility en deployments

Site Reliability

Site Reliability Engineering

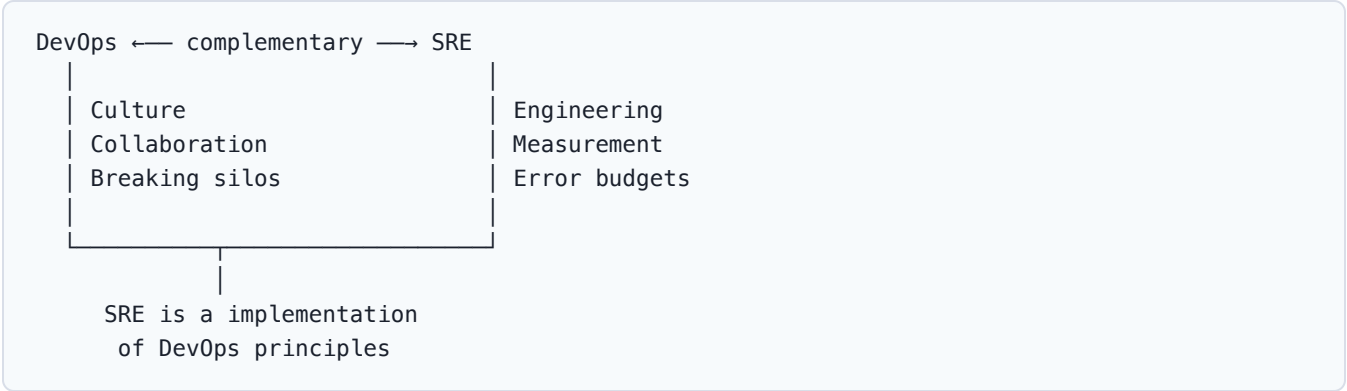
SRE es un enfoque para la ingeniería de operaciones que usa principios de software para gestionar infraestructura y sistemas, creando un marco para la confiabilidad a escala.

¿Qué es SRE?

"SRE is what happens when you treat operations as a software problem." — Google SRE Book

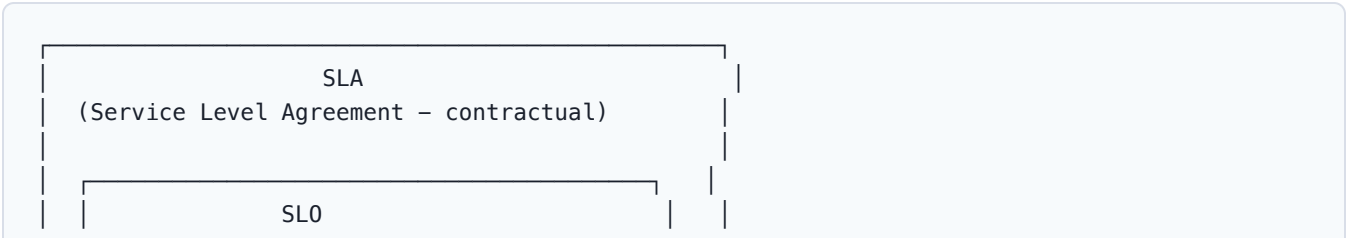
SRE vs DevOps

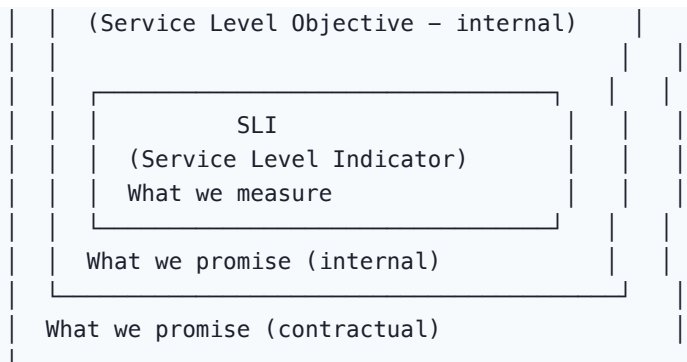
Aspecto	DevOps	SRE
Focus	Cultura y colaboración	Confiabilidad y métricas
Approach	Philosophical	Engineering-driven
Metric	DORA metrics	SLIs/SLOs/Error Budgets
Toil	Goal: reduce	Goal: eliminate (<50%)
Risk	Shift-left	Manage via error budgets



SLIs, SLOs y SLAs

Niveles de servicio





SLIs comunes

SLI	Definición	Fórmula
Availability	% de requests exitosas	successful requests / total requests
Latency	Tiempo de respuesta	requests < threshold / total requests
Throughput	Requests por segundo	total requests / time window
Error rate	% de errores	error responses / total responses
Durability	Persistencia de datos	data loss events / time period

SLO examples

```
# SLO Definition
slo:
  service: myapp
  objective: 99.9%
  sli:
    type: availability
    description: "Percentage of successful HTTP requests"
  window: 30 days
  error_budget:
    total_minutes: 43200 # 30 days
    budget_minutes: 43.2 # 0.1% of 43200
    remaining_minutes: 30
```

SLA examples

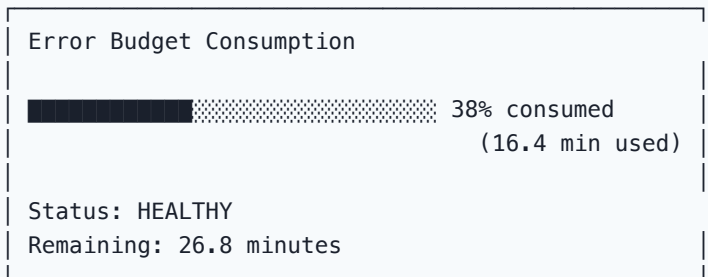
SLA Level	Availability	Downtime/year	Downtime/month
99%	"Two nines"	3.65 days	7.31 hours
99.9%	"Three nines"	8.77 hours	43.83 min

SLA Level	Availability	Downtime/year	Downtime/month
99.95%	"Three and half"	4.38 hours	21.92 min
99.99%	"Four nines"	52.6 min	4.38 min
99.999%	"Five nines"	5.26 min	26.3 sec

Error Budgets

El concepto

SL0: 99.9% availability
 Error Budget: 0.1% = 43.2 minutes/month



Error Budget Policy

Budget Status	Action
>50% remaining	Normal operations, feature velocity
25-50% remaining	Caution, more testing, slower rollouts
10-25% remaining	Freeze non-critical changes
<10% remaining	Only reliability work, no feature deploys
0% remaining	All hands on reliability, incident mode

```
# Error budget policy
policies:
  - name: healthy
    threshold: 50
    actions:
      - allow_feature_deploys: true
      - require_extra_testing: false

  - name: warning
    threshold: 25
```

```
actions:
  - allow_feature_deploys: true
  - require_extra_testing: true
  - notify_team: true

- name: critical
  threshold: 10
  actions:
    - allow_feature_deploys: false
    - require_reliability_review: true
    - escalate_to_management: true
```

Toil Reduction

"Toil is the kind of work that tends to be manual, repetitive, automatable, tactical, devoid of enduring value, and scales linearly as the service grows."

Categorías de toil

Tipo	Ejemplo	Solución
Manual ops	Restart services manually	Auto-remediation scripts
Repeated debugging	Same issue, different day	Fix root cause, alerting
Manual provisioning	Create resources by hand	IaC + self-service
Manual scaling	Resize instances	HPA, autoscaling
Toil-heavy testing	Manual regression testing	Automated test suite

Medir toil

Total engineering time: 40 hours/week
Toil time: 12 hours/week (30%)
Toil budget: < 50%

Goal: Reduce toil to <20% (8 hours/week)
Current gap: 4 hours/week to eliminate

Herramientas para medir:

- Time tracking en tickets
- Automation coverage reports
- Manual vs automated incident response

Monitoring y Alerting

Los 4 signos de黄金 signals (Google)

Signal	Descripción	SLI
Latency	Tiempo de servicio	p50, p99
Traffic	Demand on the system	RPS
Errors	Rate of failed requests	% 5xx
Saturation	How "full" is the service	CPU, memory, connections

Alerting philosophy

```
# Multi-window, multi-burn-rate alerting
# SLO: 99.9% over 30 days

alert:
  name: high_error_rate
  sli: availability
  objective: 99.9%
  conditions:
    - window: 1 hour
      burn_rate: 14.4 # 1% budget consumed in 1 hour
    - window: 5 minutes
      burn_rate: 3    # 1% budget consumed in 5 min
  severity: page # Pages on-call
```

Alert routing:

```
Alert → AlertManager → Routing
|
├─ Page (critical, needs immediate action)
├─ Ticket (non-critical, needs action soon)
└─ Log (informational, no action needed)
```

SRE Practices

Incident management

Ver Incident Response para detalles completos.

Post-mortems

Tipo	Descripción
Blameless	Focus en sistemas, no personas
Written	Documentado y compartido

Tipo	Descripción
Action items	Con owners y deadlines
Follow-up	Verificar que se implementaron

Capacity planning

Current usage: 1000 RPS

Growth rate: 20%/month

Target: 12 months

Month 6: $1000 * 1.2^6 = 2986$ RPS

Month 12: $1000 * 1.2^{12} = 8916$ RPS

Current capacity: 1500 RPS

Action needed: Scale by month 4

Relación con otros conceptos

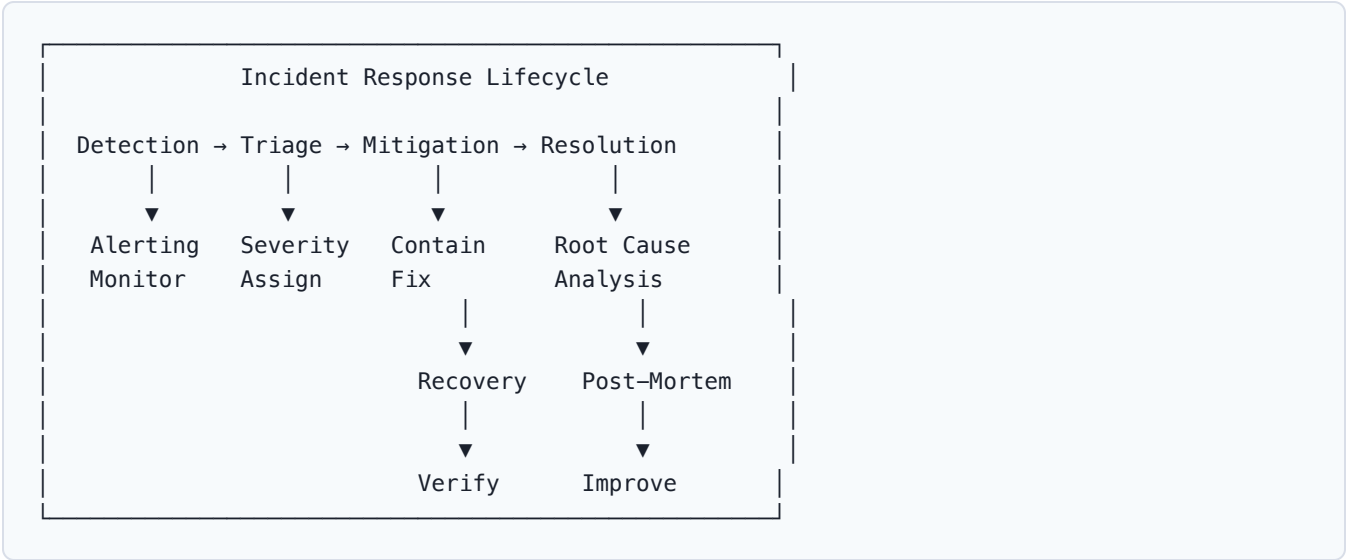
- Incident Response — Gestión de incidentes
- DORA Metrics — Métricas de delivery
- MTTR — Mean Time To Recovery
- Cloud Native — SRE en cloud
- El Ciclo de DevOps — Measurement en CALMS
- Team Health Metrics — SRE como práctica de equipo

Incident Response

Incident Response y Post-Mortems

La respuesta efectiva a incidentes es crítica para mantener la confiabilidad del sistema. Este artículo cubre el ciclo completo: desde la detección hasta la mejora post-incidente.

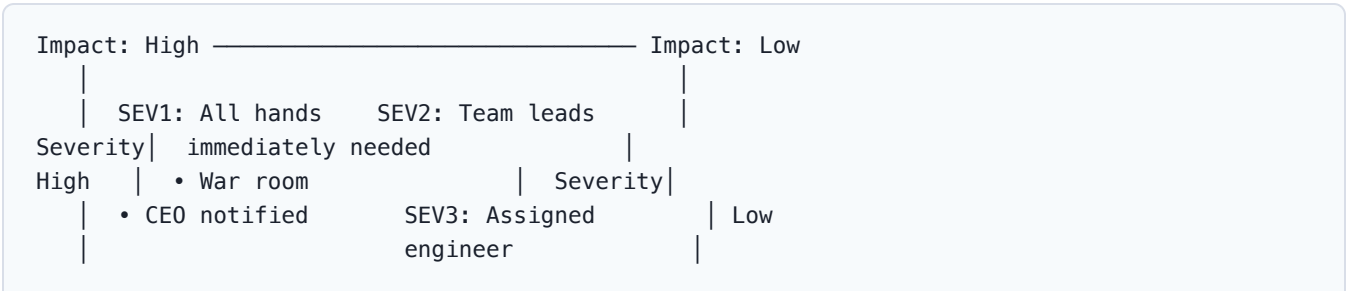
Ciclo de Incident Response



Severity Levels

Level	Nombre	Descripción	Response Time	Ejemplo
SEV1	Critical	Service down, data loss, security breach	Immediate	Production outage
SEV2	Major	Degraded performance, feature unavailable	< 30 min	Slow API responses
SEV3	Minor	Limited impact, workaround available	< 2 hours	UI bug, non-critical
SEV4	Low	Cosmetic, no user impact	< 24 hours	Typo, non-urgent

Decision matrix



SEV2: Page on-call	• No escalation
	SEV4: Backlog

On-Call

Rotation schedule

```
# PagerDuty-style rotation
rotation:
  primary:
    schedule: "1 week on / 2 weeks off"
    responders:
      - name: "Alice"
        phone: "+1-555-0101"
      - name: "Bob"
        phone: "+1-555-0102"

  secondary:
    schedule: "1 week on / 2 weeks off (offset)"
    responders:
      - name: "Charlie"
        phone: "+1-555-0103"
      - name: "Diana"
        phone: "+1-555-0104"

  escalation:
    - after: 15 min
      notify: secondary
    - after: 30 min
      notify: team-lead
    - after: 60 min
      notify: engineering-director
```

On-call best practices

Práctica	Descripción
Handoff document	Context between rotations
Runbook per alert	Step-by-step for each alert
No hero culture	Ask for help early
Compensate fairly	On-call pay or time off
Rotate regularly	Prevent burnout

Práctica	Descripción
Track load	Monitor pages per rotation

Runbooks

Estructura de un runbook

```
# Runbook: High Error Rate Alert

## Alert
Service: myapp-service
Alert: high_error_rate (>5% for 5 min)

## Impact
Users experiencing 500 errors on API endpoints.
~10% of requests failing.

## Diagnosis

### Step 1: Check service status
kubectl get pods -n production -l app=myapp

### Step 2: Check recent deployments
kubectl rollout history deployment/myapp -n production

### Step 3: Check logs
kubectl logs -n production -l app=myapp --tail=100 | grep ERROR

### Step 4: Check dependencies
curl -s http://myapp-service/health | jq .

## Mitigation

### Option A: Rollback
kubectl rollout undo deployment/myapp -n production

### Option B: Restart pods
kubectl rollout restart deployment/myapp -n production

### Option C: Scale up
kubectl scale deployment/myapp --replicas=5 -n production

## Escalation
If above steps don't resolve:
1. Page team lead
2. Open war room
3. Check downstream services

## Post-Incident
```

- [] Create post-mortem document
- [] Update runbook if needed
- [] Fix root cause

Blameless Post-Mortems

Principe fundamental

"Blameless does not mean we are blind to problems. It means we create an environment where we can discuss failures without fear of punishment."

Template de post-mortem

```
# Post-Mortem: Production Outage 2026-07-10

## Summary
- **Duration**: 47 minutes (14:23 - 15:10 UTC)
- **Impact**: 100% of API requests failing
- **Severity**: SEV1
- **Incident commander**: Alice

## Timeline
- 14:23 - Alert fires: high_error_rate
- 14:25 - On-call (Bob) acknowledges alert
- 14:28 - Bob starts diagnosis, checks pods
- 14:32 - Bob identifies database connection pool exhaustion
- 14:35 - Bob restarts service (attempts mitigation)
- 14:40 - Restart doesn't fix, escalation to Alice
- 14:45 - Alice identifies root cause: new migration caused connection leak
- 14:50 - Alice rolls back migration
- 15:00 - Database connections recover
- 15:10 - All services healthy, incident resolved

## Root Cause
Database migration v234 introduced a connection leak. Each API request opened a new connection b

## Impact
- 100% API failure for 47 minutes
- ~50,000 requests failed
- 3 customer escalations
- No data loss

## What went well
- Alert fired quickly (within 5 min of issue start)
- On-call responded within 2 minutes
- Escalation was smooth
- Rollback was clean

## What went wrong
```

- Migration wasn't caught in testing
- Connection pool monitoring wasn't alerting
- No load test for migration

Action Items

Action	Owner	Deadline	Status
Add connection pool monitoring	Bob	2026-07-17	Open
Add migration testing to CI	Charlie	2026-07-24	Open
Load test all migrations	Diana	2026-07-31	Open
Fix connection leak in migration	Alice	2026-07-14	Done

Lessons Learned

- Migrations need connection pool testing
- Monitoring should cover all critical resources
- Small changes can have big impacts

Post-mortem principles

Principle	Descripción
Blameless	Focus on systems, not people
Written	Documented and shared
Data-driven	Use metrics and timelines
Action-oriented	Every finding has an action item
Follow-up	Verify actions were completed
Time-boxed	Complete within 1 week

Communication during incidents

Internal communication

Incident Channel: #inc-2026-07-10-db-leak

@on-call: Investigating high error rate on API
@on-call: Found database connection pool exhaustion
@lead: Escalating, joining war room
@lead: Root cause identified: migration v234
@lead: Rolling back migration
@on-call: Rollback complete, monitoring recovery
@lead: All services healthy. Incident resolved.
@lead: Post-mortem scheduled for tomorrow 10am

External communication

Status Page Update

[14:23] Investigating

We are investigating reports of API errors.
Some users may experience 500 errors.

[14:45] Identified

We have identified the issue as a database problem.
Working on a fix.

[15:10] Resolved

The issue has been resolved. All API services
are operating normally. We will provide a
post-mortem within 48 hours.

Incident metrics

Métrica	Descripción	Objetivo
MTTD	Mean Time To Detect	< 5 min
MTTA	Mean Time To Acknowledge	< 2 min
MTTM	Mean Time To Mitigate	< 30 min
MTTR	Mean Time To Resolve	< 1 hour
MTBF	Mean Time Between Failures	> 30 days

Relacionado con MTTR.

Relación con otros conceptos

- SRE — Framework para confiabilidad
- MTTR — Métrica clave de incidentes
- Gestión de Incidentes — Proceso formal
- El Ciclo de DevOps — Sharing en CALMS
- DevSecOps — Security incidents
- Team Health Metrics — Post-mortem como mejora

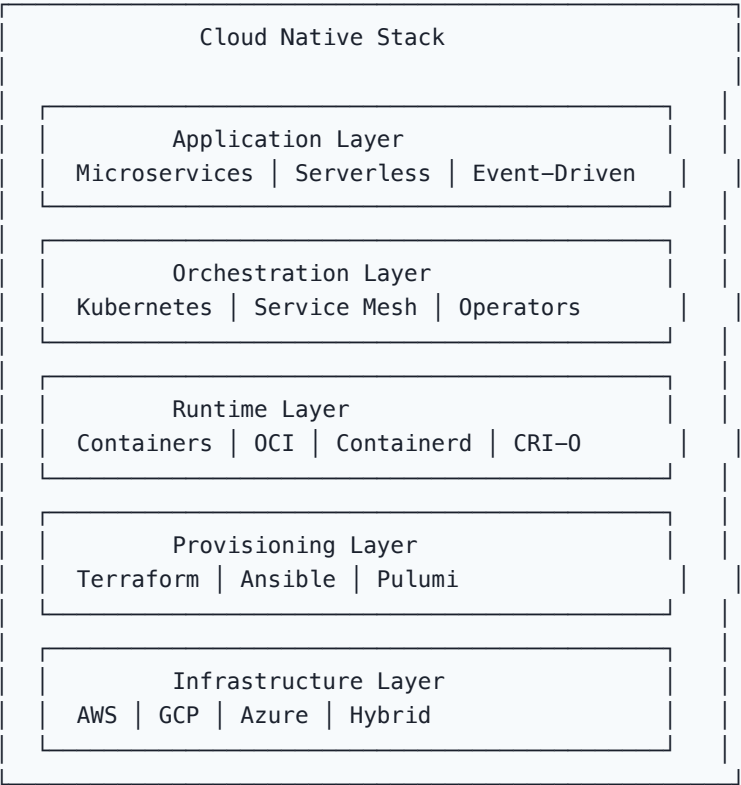
Cloud Native

Cloud Native Architecture

Cloud Native es un enfoque para construir y ejecutar aplicaciones que aprovechan completamente las ventajas del modelo de computing en la nube.

¿Qué es Cloud Native?

"Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds." — CNCF



Las 4 Columnas de Cloud Native

Columna	Descripción	Herramientas
Containers	Packaging y runtime	Docker, containerd, Podman
Microservices	Arquitectura distribuida	REST, gRPC, Events
Service Mesh	Networking entre servicios	Istio, Linkerd, Consul

Columna	Descripción	Herramientas
Declarative APIs	Configuración como código	Kubernetes, Terraform

The Twelve-Factor App

#	Factor	Descripción
1	Codebase	One codebase in version control, many deploys
2	Dependencies	Explicitly declare and isolate dependencies
3	Config	Store config in the environment
4	Backing services	Treat backing services as attached resources
5	Build, release, run	Strictly separate build and run stages
6	Processes	Execute the app as one or more stateless processes
7	Port binding	Export services via port binding
8	Concurrency	Scale out via the process model
9	Disposability	Maximize robustness with fast startup and graceful shutdown
10	Dev/prod parity	Keep development, staging, and production as similar as possible
11	Logs	Treat logs as event streams
12	Admin processes	Run admin/management tasks as one-off processes

Implementación práctica

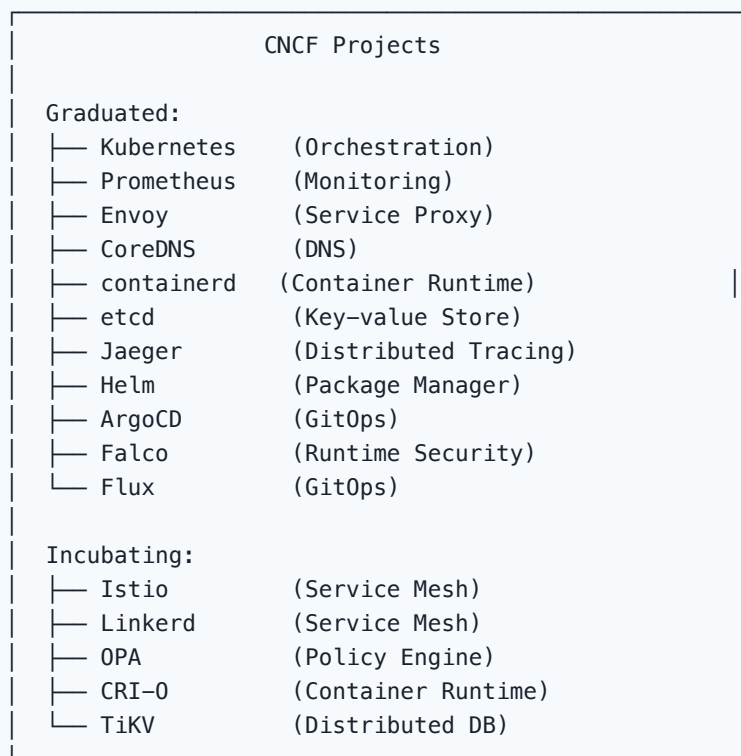
```
# Factor 3: Config in environment
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      containers:
        - name: app
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: db-secret
                  key: url
            - name: LOG_LEVEL
              valueFrom:
                configMapKeyRef:
                  name: app-config
```

```
key: log_level

# Factor 7: Port binding
ports:
  - containerPort: 8080

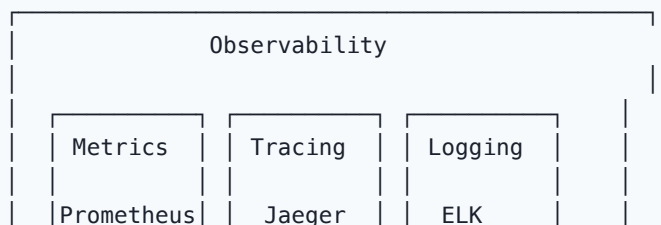
# Factor 6: Stateless processes
env:
  - name: SESSION_STORE
    value: "redis://redis:6379"
```

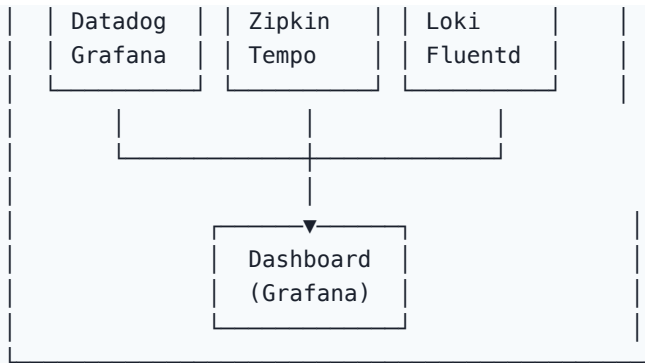
CNCF Landscape



Observability

Las 3 pilas de observabilidad





OpenTelemetry

```

// OpenTelemetry instrumentation
const { NodeTracerProvider } = require('@opentelemetry/sdk-trace-node');
const { JaegerExporter } = require('@opentelemetry/exporter-jaeger');
const { PrometheusExporter } = require('@opentelemetry/exporter-prometheus');

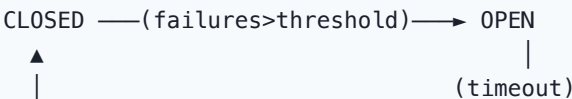
const provider = new NodeTracerProvider();
const jaegerExporter = new JaegerExporter({
  serviceName: 'my-service',
});
const prometheusExporter = new PrometheusExporter({ port: 9464 });

provider.addSpanProcessor(new BatchSpanProcessor(jaegerExporter));
provider.register();
  
```

Resilience Patterns

Pattern	Descripción
Circuit Breaker	Stop calling failing service
Retry	Retry with exponential backoff
Timeout	Don't wait forever
Bulkhead	Isolate failures
Rate Limiting	Protect from overload
Fallback	Graceful degradation

Circuit Breaker States:



| (success) — HALF-OPEN ← |

Cloud Native vs Traditional

Aspecto	Traditional	Cloud Native
Architecture	Monolithic	Microservices
Deployment	VMs, manual	Containers, automated
Scaling	Vertical	Horizontal
State	Stateful	Stateless
Failure	Prevent all	Embrace failure
Release	Months	Hours/minutes
Infrastructure	Pets	Cattle

Relación con otros conceptos

- Containers — Building blocks
- Kubernetes — Orchestration platform
- Service Mesh — Networking layer
- Microservicios — Architecture pattern
- Arquitectura Limpia — Design principles
- GitOps — Deployment strategy
- Serverless — Next evolution

Serverless Edge

Serverless y Edge Computing

Serverless elimina la gestión de infraestructura, permitiendo enfocarse exclusivamente en el código. Edge Computing ejecuta lógica cerca del usuario final para reducir latencia.

¿Qué es Serverless?

Traditional:

Provision
servers
(always on)

You manage:

- OS
- Runtime
- Scaling
- Patching

Serverless:

Write code
(on-demand)

Provider manages:

- Servers
- Scaling
- Patches

Modelos serverless

Modelo	Descripción	Ejemplo
FaaS	Functions as a Service	AWS Lambda, Azure Functions
BaaS	Backend as a Service	Firebase, Supabase
Serverless containers	Containers on-demand	AWS Fargate, Cloud Run
Serverless databases	Auto-scaling DBs	DynamoDB, PlanetScale

AWS Lambda

Function example

```
// Simple Lambda function
exports.handler = async (event) => {
  const body = JSON.parse(event.body);

  // Business logic
  const result = await processOrder(body);

  return {
```

```

        statusCode: 200,
        headers: {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        },
        body: JSON.stringify({
            message: 'Order processed',
            orderId: result.id
        })
    };
};

async function processOrder(order) {
    // Validate, save to DB, send email, etc.
    const orderId = generateId();

    await saveToDynamoDB({
        orderId,
        items: order.items,
        total: calculateTotal(order.items),
        status: 'confirmed',
        createdAt: new Date().toISOString()
    });

    await sendEmail(order.email, `Order ${orderId} confirmed`);

    return { id: orderId };
}

```

Lambda + API Gateway

```

Client → API Gateway → Lambda → DynamoDB
                        |
                        ├── Lambda (auth)
                        ├── Lambda (business logic)
                        └── Lambda (notifications)

```

```

# SAM Template
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31

Globals:
  Function:
    Timeout: 30
    Runtime: nodejs18.x
    MemorySize: 128
    Environment:
      Variables:
        TABLE_NAME: !Ref OrdersTable

```

```

Resources:
  ApiFunction:
    Type: AWS::Serverless::Function
    Properties:
      Handler: src/handlers/api.handler
      Events:
        Api:
          Type: Api
          Properties:
            Path: /{proxy+}
            Method: ANY
            RestApiId: !Ref Api

  OrdersTable:
    Type: AWS::DynamoDB::Table
    Properties:
      TableName: orders
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions:
        - AttributeName: orderId
          AttributeType: S
      KeySchema:
        - AttributeName: orderId
          KeyType: HASH

```

Cloudflare Workers

Edge Computing

```

// Cloudflare Worker at the edge
addEventListener('fetch', event => {
  event.respondWith(handleRequest(event.request));
});

async function handleRequest(request) {
  // Runs at the edge, close to the user
  const url = new URL(request.url);

  if (url.pathname === '/api/time') {
    return new Response(JSON.stringify({
      timezone: Intl.DateTimeFormat().resolvedOptions().timeZone,
      time: new Date().toISOString(),
      region: request.cf?.colo // Cloudflare data center
    }), {
      headers: { 'Content-Type': 'application/json' }
    });
  }
}

```

```
// Proxy with caching
const response = await fetch(request);
const newResponse = new Response(response.body, response);
newResponse.headers.set('Cache-Control', 'public, max-age=300');

return newResponse;
}
```

Workers vs Lambda@Edge

Aspecto	Cloudflare Workers	Lambda@Edge
Cold start	< 5ms	50-500ms
Runtime	V8 isolates	Node.js
Limits	30s CPU, 128MB	30s, 128MB
Pricing	Per request + CPU time	Per request + duration
Global	300+ locations	200+ locations
K/V Store	Built-in (Workers KV)	Not built-in

Casos de uso

Serverless scenarios

Scenario	Solution
API endpoints	Lambda + API Gateway
Data processing	Lambda + S3 events
Scheduled tasks	EventBridge + Lambda
Real-time streaming	Kinesis + Lambda
Chatbots	Lex + Lambda
IoT processing	IoT Core + Lambda

Edge scenarios

Scenario	Solution
Personalization	Workers modify response based on user
A/B testing	Route to different origins at edge
Bot detection	Block malicious traffic at edge

Scenario	Solution
Image optimization	Resize/transform images at edge
Geolocation	Serve content based on location
Rate limiting	Protect APIs at edge

Tradeoffs de Serverless

Ventajas

Ventaja	Descripción
No ops	Sin servers que gestionar
Auto-scaling	Escala a 0 y a millones
Pay per use	Solo pagas por ejecución
Fast startup	Deploy en segundos
Focus on code	Sin infra, solo lógica

Desventajas

Desventaja	Descripción
Cold starts	Latencia en primera invocación
Vendor lock-in	Tightly coupled al provider
Limits	Memory, time, concurrency limits
Debugging	Más difícil que applications tradicionales
Cost at scale	A gran escala puede ser más caro

Cost comparison:

Low traffic (< 1M req/month):

Serverless: ~\$5/month ✓

Containers: ~\$50/month (minimum)

Medium traffic (10M req/month):

Serverless: ~\$50/month

Containers: ~\$100/month

High traffic (100M+ req/month):

Serverless: ~\$500/month
Containers: ~\$200/month ✓ (optimize with reserved)

Comparación de plataformas

Platform	Cold Start	Free Tier	Pricing Model
AWS Lambda	100-500ms	1M req/month	Per request + duration
Azure Functions	100-500ms	1M req/month	Per request + duration
Google Cloud Functions	100-500ms	2M req/month	Per request + duration
Cloudflare Workers	< 5ms	100K req/day	Per request + CPU time
Deno Deploy	< 10ms	100K req/day	Per request

Serverless + Containers



Relación con otros conceptos

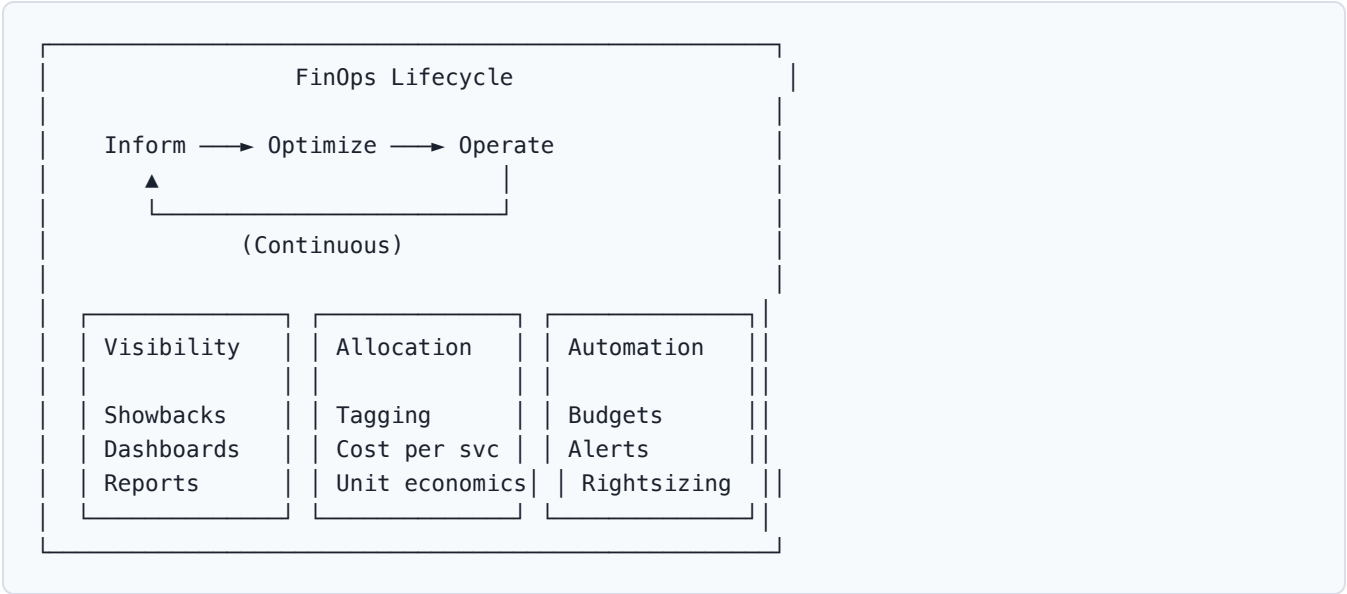
- Cloud Native — Serverless es cloud native
- Containers — Alternativa/complemento a serverless
- Optimización de Costos — Pricing models
- Plataformas Cloud — Providers
- Software Categoría A — When serverless is appropriate
- IaC — Infrastructure for serverless

Cost Optimization

Optimización de Costos en Cloud

FinOps es el framework operativo que trae la responsabilidad financiera al consumo de cloud, permitiendo a los equipos tomar decisiones basadas en datos de costo.

¿Qué es FinOps?



Pilares de optimización

1. Right-sizing

Before (over-provisioned):

m5.2xlarge
8 vCPU, 32GB RAM
\$221/month

CPU avg: 5%
Memory avg: 15%

→

After (right-sized):

t3.medium
2 vCPU, 4GB RAM
\$30/month

CPU avg: 45%
Memory avg: 60%

Savings: \$191/month (86%)

Estrategia	Ahorro	Riesgo
Right-sizing	30-60%	Bajo

Estrategia	Ahorro	Riesgo
Reserved Instances	40-70%	Medio (commitment)
Spot Instances	60-90%	Alto (interrupción)
Scheduling	65% (off-hours)	Bajo
Graviton/ARM	20%	Bajo

2. Reserved Instances y Savings Plans

Instance Pricing Models	
On-Demand:	\$0.10/hr (no commitment)
Reserved:	\$0.06/hr (1yr commitment) \$0.04/hr (3yr commitment)
Spot:	\$0.03/hr (can be interrupted)
Savings Plan:	Flexible commitment \$0.05/hr (1yr, \$10k commit)

Estrategia de cobertura:

Total usage: 100%

- └ Reserved/Savings Plans: 70% (base load)
- └ On-Demand: 25% (variable)
- └ Spot: 5% (fault-tolerant workloads)

3. Spot Instances

```
# Kubernetes Spot Instances
apiVersion: apps/v1
kind: Deployment
metadata:
  name: batch-processor
spec:
  replicas: 10
  template:
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
```

```

      - matchExpressions:
        - key: node.kubernetes.io/capacity-type
          operator: In
          values: ["spot"]
    tolerations:
      - key: "spot"
        operator: "Equal"
        value: "true"
        effect: "NoSchedule"
    containers:
      - name: processor
        image: batch-processor:latest

```

Cuándo usar spot:

Workload	Spot suitable?
Batch processing	✓ Yes
CI/CD runners	✓ Yes
Development environments	✓ Yes
Fault-tolerant apps	✓ Yes
Stateful services	✗ No
Mission-critical	✗ No
Real-time serving	⚠ Maybe with fallback

4. Scheduling

Work schedule:

Mon-Fri: 8am-8pm	Scale UP
Mon-Fri: 8pm-8am	Scale DOWN
Weekends: ALL DAY	Scale DOWN
Holidays: ALL DAY	Scale DOWN

```

# Cron-based autoscaling
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: work-hours-scaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment

```

```

    name: myapp
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: External
      external:
        metric:
          name: queue_messages_ready
        target:
          type: AverageValue
          averageValue: "10"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 25
          periodSeconds: 120

```

Unit Economics

Cost per transaction

```

# Cost per user
total_cloud_cost = 10000 # monthly
total_users = 100000 # monthly active users
cost_per_user = total_cloud_cost / total_users
# $0.10 per user per month

# Cost per transaction
total_transactions = 5000000
cost_per_transaction = total_cloud_cost / total_transactions
# $0.002 per transaction

# Cost as % of revenue
monthly_revenue = 500000
cost_percentage = (total_cloud_cost / monthly_revenue) * 100
# 2% of revenue (healthy: <10%)

```

Cost allocation tags

```
# Required tags for cost allocation
tags:
  Environment: production
  Team: payments
  Service: checkout-api
  CostCenter: engineering
  Project: checkout-v2
  ManagedBy: terraform
```

Herramientas de FinOps

Tool	Type	Features
AWS Cost Explorer	Native	Recommendations, forecasting
AWS Compute Optimizer	Native	Right-sizing suggestions
Kubecost	K8s	Cost per namespace, pod
CloudHealth	Multi-cloud	Showback, optimization
Infracost	IaC	Cost estimates in PRs
OpenCost	CNCF	K8s cost allocation

```
# Infracost - Cost estimates in PRs
$ infracost diff --path .

Project: myproject

+ aws_instance.web
+$45.60/mo

~ aws_instance.api (t3.large → t3.medium)
-$22.80/mo

Monthly cost change: -$22.80 (-15%)
```

Alertas de costos

```
# Budget alerts
budgets:
  - name: "Monthly budget"
    amount: 10000
    alerts:
      - threshold: 80
        notify: ["finance@company.com", "eng-leads@company.com"]
      - threshold: 100
```

```
notify: ["cto@company.com"]  
action: "notify-only"
```

- name: "Unexpected spike"
type: "anomaly"
threshold: 150 # 150% of daily average
notify: ["on-call@company.com"]

Checklist de optimización

Área	Acción	Ahorro potencial
Compute	Right-size instances	30-60%
Storage	Lifecycle policies (S3)	40-70%
Database	Reserved capacity	40-60%
Network	CDN for static assets	20-50%
Containers	Spot for non-critical	60-90%
Dev/Staging	Auto-shutdown off-hours	65%
Images	Delete old snapshots	20-40%
Logs	Retention policies	30-50%

Relación con otros conceptos

- Cloud Native — Cloud cost visibility
- IaC — Cost estimates in code
- Serverless — Pay-per-use pricing
- Business Value Metrics — Cost vs value
- Plataformas Cloud — Provider comparisons
- SRE — Capacity planning

README

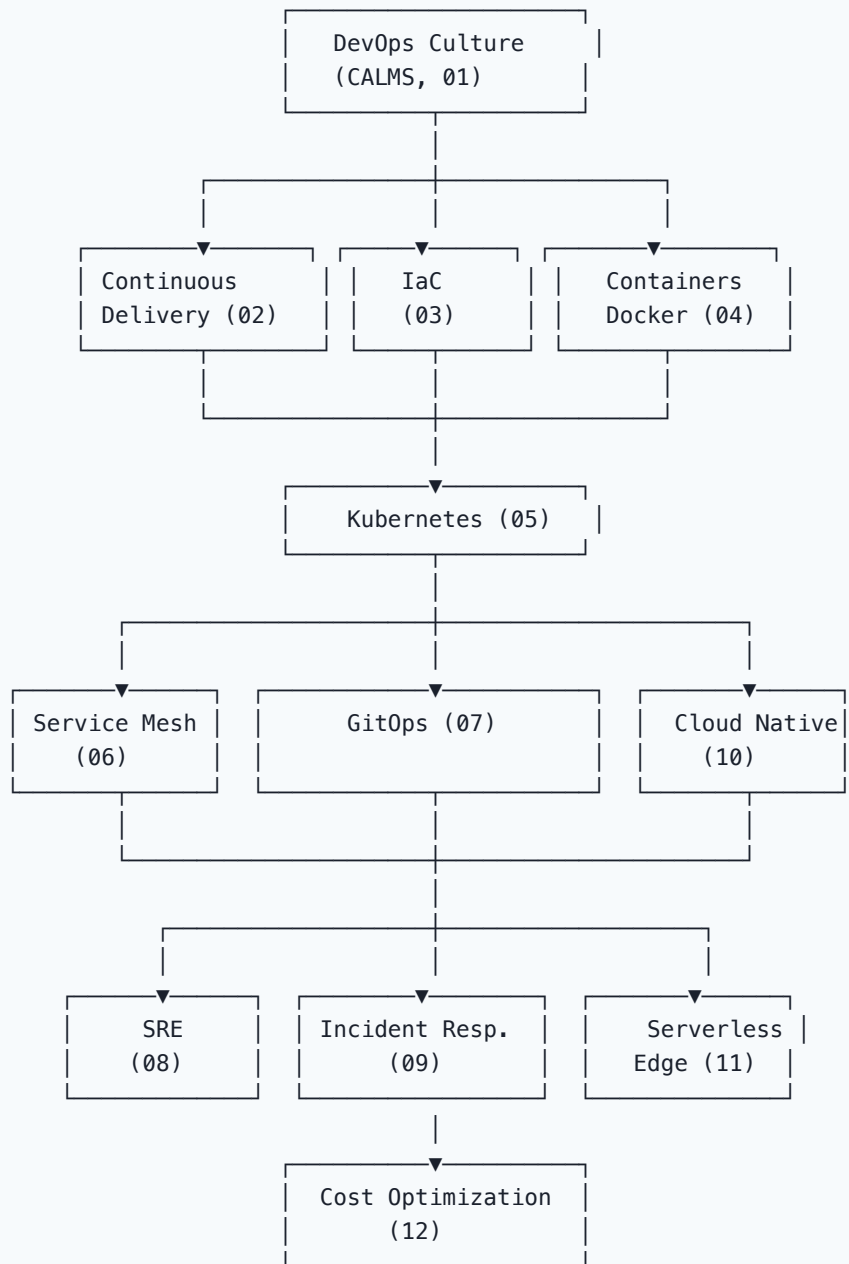
DevOps e Infraestructura

Prácticas, herramientas y arquitecturas para entregar software de forma continua, escalable y resiliente. Desde los fundamentos de DevOps hasta la optimización de costos en cloud.

Contenido

#	Archivo	Tema	Nivel	
01	[[../09-DevOps-Infra/01-devops-ciclo	El Ciclo de DevOps]]	Infinite loop, CALMS, cultura DevOps	Principiante
02	[[../09-DevOps-Infra/02-continuous-delivery	Continuous Delivery]]	Principios, deployment pipeline, Jez Humble	Intermedio
03	[[../09-DevOps-Infra/03-infrastructure-as-code	Infrastructure as Code]]	Terraform, Pulumi, Ansible, state management	Intermedio
04	[[../09-DevOps-Infra/04-containers-docker	Containers y Docker]]	Dockerfile, compose, networking, volumes	Principiante
05	[[../09-DevOps-Infra/05-kubernetes-orchestration	Kubernetes y Orquestación]]	Pods, services, deployments, Helm, operators	Avanzado
06	[[../09-DevOps-Infra/06-service-mesh	Service Mesh]]	Istio, Linkerd, sidecar pattern, mTLS	Avanzado
07	[[../09-DevOps-Infra/07-gitops	GitOps]]	Principios, ArgoCD, Flux, GitOps vs CI/CD	Intermedio
08	[[../09-DevOps-Infra/08-site-reliability	Site Reliability Engineering]]	SLIs/SLOs, Error Budgets, toil reduction	Avanzado
09	[[../09-DevOps-Infra/09-incident-response	Incident Response]]	Severity, on-call, runbooks, post-mortems	Intermedio
10	[[../09-DevOps-Infra/10-cloud-native	Cloud Native Architecture]]	12-factor app, CNCF, microservices, observability	Intermedio
11	[[../09-DevOps-Infra/11-serverless-edge	Serverless y Edge Computing]]	Lambda, Cloudflare Workers, use cases	Intermedio
12	[[../09-DevOps-Infra/12-cost-optimization	Optimización de Costos]]	FinOps, rightsizing, reserved instances, spot	Intermedio

Mapa conceptual



Secciones relacionadas

- 01 — Fundamentos — Filosofía DevOps, shift-left, lean
- 03 — Procesos — CI/CD pipeline, trunk-based development
- 05 — Herramientas — Terraform, containers, monitoring
- 06 — Métricas — DORA metrics, MTTR, deployment frequency
- 08 — Calidad y Seguridad — DevSecOps, quality gates